Teil II

Datenstrukturen und Algorithmen

0.9 Einleitung

0.9.1 Literatur und Symbolverzeichnis

0.9.1.1 Literatur zu Algorithmen und Datenstrukturen

- Ralf-Hartmut Güting, Stefan Dieker
  Datenstrukturen und Algorithmen
  Wesentliche Grundlage dieses 2. Skriptteils.

- Thomas H. Cormen, Charles E. Leiserson, Ronald Rivest, Clifford Stein
  Algorithmen – Eine Einführung
  Referenzwerk, sehr umfangreich.

© R.-R. Grigat 2003-2019

0.9.1.2 Symbolverzeichnis

a_i, b_i	Listenelemente
c	Ereignis, Wert, Zeichen
C	Ereignismenge, Wertemenge
d	Knotengrad, Baumgrad
d_s	Daten pro Seite
e, e_i	Element
$E\{T\}$	Erwartungswert der Kosten für Suche in Baum T
f_1, f_2	Sequentielle Eingangsdaten
g_1, g_2	Sequentielle Ausgangsdaten
h	Baumhöhe, Hashadresse
H	Heap
i	Index
k	Schlüsselwert, Schlüssel
l	Pfadlänge, Codewortlänge, Index für linken Teilbaum
L	Linker Teilbaum im Binärbaum
m	Schlüssel pro Knoten, Minimum, Behälterzahl in Hashtabelle, Mächtigkeit einer Menge
n	Anzahl behandelter Daten, Problemgröße
N	Mächtigkeit des Datentyps der Eingangsdaten
O	Komplexität
p, p_e	Knoten, neuer Knoten für Element e
p_l, p_r, p_v	Knoten von Element, linkem und rechtem Sohn, Vater
p_i	Zeiger auf Listenelement (in Datenstruktur)
P	Wahrscheinlichkeit
q	Queue
q_p	Priority Queue
r	Index für rechten Teilbaum
Q	Quellfolge
R	Rechter Teilbaum im Binärbaum
s	Anzahl Schlüssel in Knoten, Anzahl Seiten
S	Array, Suchfolge
T, T_i	Teilbaum

Tabelle 9: Symbolverzeichnis

© R.-R. Grigat 2003-2019

0.9.2 Algorithmen und Datenstrukturen

0.9.2.1 Abstraktionsebenen von Algorithmen und Datenstrukturen

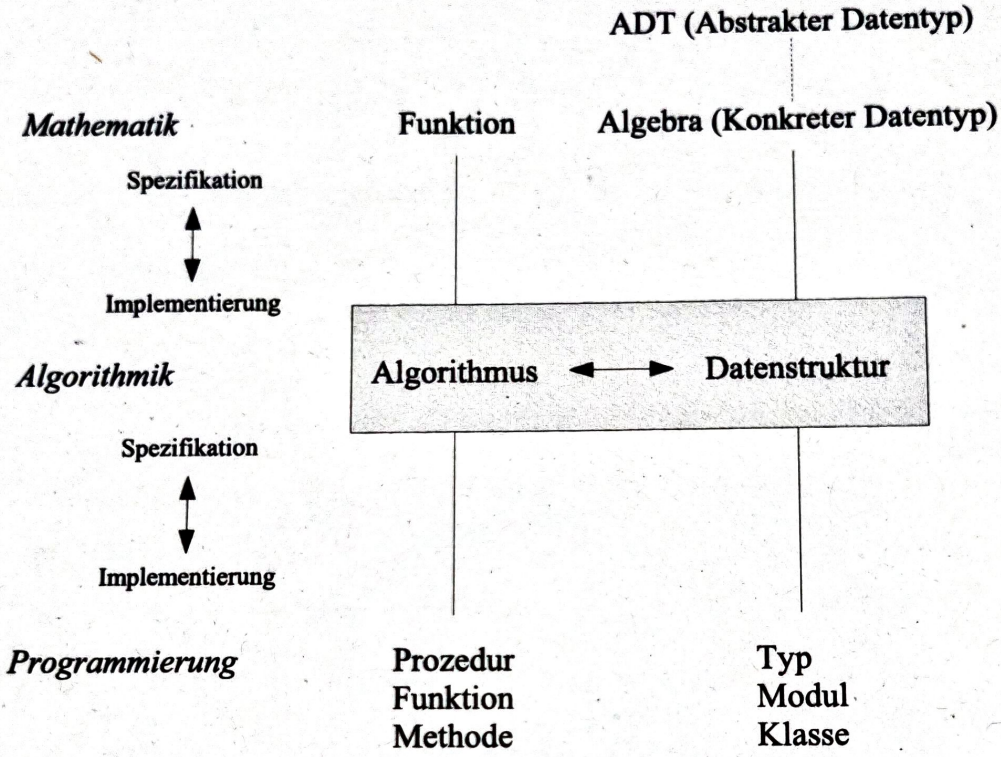


Abbildung 51: Abstraktionsebenen nach Güting

0.9.2.2 Abstraktionsebenen

Abstraktionsebenen

- Mathematik
- Algorithmik
- Programmierung

Abstraktionsebene: Mathematik

- Präzise Beschreibung, *was* berechnet wird
- Offenlassen *wie* etwas berechnet wird

Abstraktionsebene: Algorithmik

- Beschreibung für *Menschen*, *wie* etwas berechnet wird
- Beschreibung nicht für Compiler gedacht
- Pseudocode üblich mit geeigneter Abstraktion
- Unabhängigkeit von Details einer Programmiersprache

Abstraktionsebene: Programmierung

- Beschreibung eines Algorithmus für Compiler
- Konkrete Implementierung in bestimmter Programmiersprache

Abstraktionsebenen für *Algorithmen*

- Mathematische Funktion
  - math. Spezifikation eines Algorithmus
- Algorithmus
  - Algorithmische Implementierung math. Funktion
  - Algor. Spezifikation von Prozedur (Methode)
- Prozedur, Funktion, Methode

© R.-R. Grigat 2003-2019

- Implementierung eines Algorithmus in Programmiersprache

Abstraktionsebenen für *Datenstrukturen*

- Abstrakter Datentyp (ADT)
  - Trägermengen und Operationen anhand *Axiomen* charakterisieren
  - nur *interessierende* Aspekte angegeben
  - *automatische* Erzeugung von Prototyp
  - schwierig: Beweis der *Vollständigkeit* und *Widerspruchsfreiheit*
  - schwierig: *intuitives Verständnis* eines ADT
- Algebra (Konkreter Datentyp)
  - eine oder mehrere Objektklassen einschl. Operationen
  - *universale* Algebra: nur eine Objektklasse (z. B. natürliche Zahlen mit Addition, Multiplikation)
  - *mehrsortige* oder *heterogene* Algebra: mehrere Objektklassen (z. B. Vektorräume mit Vektoren und reellen Zahlen)
- Datenstruktur
  - Implementierung einer Algebra oder eines ADT auf algorithmischer Ebene
  - Charakterisierung anhand Signatur und verbal informaler Beschreibung
- Typ, Modul, Klasse
  - Implementierung einer Algebra oder eines ADT in einer Programmiersprache

0.9.2.3 Beziehung zwischen Algorithmus und Datenstruktur

Beispiel

- Funktion: Suche Element in Liste
- Algorithmus: Durchlaufe Liste bis Element gefunden
- Prozedur: C++ Implementierung

© R.-R. Grigat 2003-2019

- Komplexität: $O(n)$

Anmerkung:

- Falls Liste sortiert wäre, könnte binäre Suche benutzt werden:
  1. Prüfe mittleres Element
  2. Falls gesuchtes Element: fertig.
  3. Falls zu groß, benutze linke Teilliste im nächsten Schritt, weiter mit 1
  4. Falls zu klein, benutze rechte Teilliste im nächsten Schritt, weiter mit 1
- Komplexität: $O(\log n)$

$\Rightarrow$ Datenstruktur und Algorithmus als Einheit betrachten

0.9.2.4 Spezifikation einer Datenstruktur

2 Möglichkeiten:

1. Spezifikation als Algebra (konkreter Datentyp)
   - Zu jeder Objektmenge (z. B. `int`) geeignete Trägermenge (z. B. $\{M \subset \mathbb{Z} | M \text{ endlich}\}$) definieren
   - Zu jeder Operation (mathematische) Beschreibung in Form einer Funktion liefern, mit Definitions- und Wertebereich
   - Nachteil: Möglicherweise müssen Aspekte der Datenstruktur festgelegt werden, die man eigentlich nicht festlegen wollte
2. Spezifikation als abstrakter Datentyp
   - Es werden keine Trägermengen festgelegt
   - Operationen werden nicht explizit angegeben
   - Wirkungsweise von Operationen wird anhand von Gesetzen (Axiomen) charakterisiert

© R.-R. Grigat 2003-2019

0.9.2.5 Beispiel zu Abstraktionsebenen von Algorithmen

Problem:

- Finde Minimum m in einer Menge S von natürlichen Zahlen.

Spezifikation als Funktion:
Sei $\mathbb{N}$ die Menge der natürlichen Zahlen. $F(\mathbb{N})$ sei die Menge aller endlichen Teilmengen von $\mathbb{N}$. Wir definieren:

$$\min : F(\mathbb{N}) \to \mathbb{N}$$
$$\min(S) := m \quad \text{mit} \quad m \in S,\ S \in F(\mathbb{N}),\ m \leq x\ \forall\, x \in S$$

Spezifikation als Algorithmus in Pseudocode:

```
1  algorithm min(S)
2  // S ist Array der Länge n.
3  unsigned int m;
4  initalisiere m mit S[1]
5  for i:=2 to n do
6    if S[i] < m then m:= S[i] fi
7  rof
8  return m.
```

Spezifikation in Programmiersprache (z. B. C++):

```
1  int min(vector<unsigned int>& S)
2  { // S ist als vector realisiert
3    unsigned int m = S[0];
4    vector<unsigned int>::iterator i;
5    for (i = ++S.begin(); i!= S.end(); i++)
6    {
7      if (*i < m) m = *i;
8    }
9    return m;
10 }
```

© R.-R. Grigat 2003-2019

0.10 Grundlegende Datentypen

0.10.1 Listen (Folgen, Sequenzen)

0.10.1.1 Listen und Konkatenation

- *Liste* mit n Elementen: $< a_1, \ldots, a_n >$ oder alternativ (ohne Klammern und Kommata) $a_1 \ldots a_n$
- Duplikate erlaubt
- Leere Liste: $\diamond$
- Konkatenationsoperator $\circ$: $< a_1, \ldots, a_n > \circ < b_1, \ldots, b_n >$ $:= < a_1, \ldots, a_n, b_1, \ldots, b_n >$

Beispiel: KP33_3_listReverseSortSpliceMergeUnique

0.10.1.2 Modell einer einfachen Liste

Datentyp: einfache Liste

Funktionen (Beschreibung):

- *new* erzeugt leere Liste
- *first* liefert erstes Element
- *rest* liefert um erstes Element reduzierte Liste
- *append* fügt vorne Element in Liste ein
- *concat* verkettet zwei Listen
- *isempty* prüft, ob Liste leer ist

Datentypen:

$$\begin{aligned} &\textit{bool} \\ &\textit{elemSet} \\ &\textit{list} \quad = \quad \{< a_1, \ldots, a_n > | n \geq 0, \underset{1 \leq i \leq n}{\forall} a_i \in \textit{elemSet}\} \end{aligned}$$

© R.-R. Grigat 2003-2019

Funktionen: (Definition, erste Zeile jeweils Signatur)

$$
\begin{array}{lcl}
\textit{new:} & \rightarrow & \textit{list} \\
\textit{new} & = & \diamond \\
 & & \\
\textit{first: list} & \rightarrow & \textit{elemSet} \\
\textit{first}(< a_1,\ldots,a_n >) & = & \begin{cases} a_1 & \text{falls } n > 0 \\ \text{undefiniert} & \text{sonst} \end{cases} \\
\textit{rest: list} & \rightarrow & \textit{list} \\
\textit{rest}(< a_1,\ldots,a_n >) & = & \begin{cases} < a_2,\ldots,a_n > & \text{falls } n > 0 \\ \text{undefiniert} & \text{sonst} \end{cases} \\
\textit{append: list} \times \textit{elemSet} & \rightarrow & \textit{list} \\
\textit{append}(< a_1,\ldots,a_n >, e) & = & < e, a_1,\ldots,a_n > \\
 & & \\
\textit{concat: list} \times \textit{list} & \rightarrow & \textit{list} \\
\textit{concat}(< a_1,\ldots,a_n >, & & \\
< b_1,\ldots,b_n >) & = & < a_1,\ldots,a_n, b_1,\ldots,b_n > \\
 & & \\
\textit{isempty: list} & \rightarrow & \textit{bool} \\
\textit{isempty}(< a_1,\ldots,a_n >) & = & (n = 0)
\end{array}
$$

0.10.1.3 Modell einer Liste mit expliziten Positionen

Datentyp: Liste mit Positionen

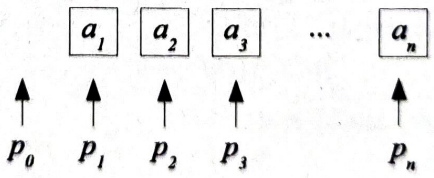


Abbildung 52: Zeiger als Positionswerte (erste Idee)

Funktionen (Beschreibung):

- p_0 zeigt auf Position vor erstem Element
  - Elemente werden hinter angegebener Position eingefügt
  - *insert*(p_0) fügt Element an erster Position ein
- *new* erzeugt leere Liste

© R.-R. Grigat 2003-2019

- *front* liefert Position vor dem ersten Element
- *last* liefert Position des letzten Elements
- *next* liefert Position des nächsten Elements
- *previous* liefert Position des vorhergehenden Elements
- *bol* („begin of list“) stellt fest, ob gegebene Position erste ist
- *eol* („end of list“) stellt fest, ob gegebene Position letzte ist
- *insert* fügt Element hinter gegebener Position ein
- *delete* löscht Element an gegebener Position
- *concat* verkettet zwei Listen
- *isempty* prüft, ob Liste leer ist
- *find* liefert Position des ersten Elements, für das die Parameterfunktion f wahr ist
- *retrieve* liefert Element an gegebener Position

Datentypen:

$$
\begin{array}{lcl}
\textit{bool} & & \\
\textit{elemSet} & = & \text{Menge gültiger Elemente} \\
\textit{pos} & = & \text{Menge gültiger Elementadressen ohne } \textit{null} \\
\textit{list} & = & \{(<a_1,\ldots,a_n>,<p_0,\ldots,p_n>) | n \in \mathbb{N} \cup \{0\}, \\
 & & \quad \forall_{i,j \in 1,\ldots,n}\ a_i \in \textit{elemSet}, \\
 & & \quad \forall_{i,j \in 0,\ldots,n}\ p_i \in \textit{pos}, \\
 & & \quad p_i \neq p_j \text{ falls } i \neq j\}
\end{array}
$$

© R.-R. Grigat 2003-2019

Funktionen (Definition):
Sei $l = (< a_1, \ldots, a_n >, < p_0, \ldots, p_n >)$ Liste und f Funktion.

$$
\begin{array}{lcl}
\textit{new:} & \to & \textit{list} \\
\textit{new} & = & (\diamond, < p_0 >)
\end{array}
$$

$$
\begin{array}{lcl}
\textit{front:}\quad \textit{list} & \to & \textit{pos} \\
\textit{front}(l) & = & p_0
\end{array}
$$

$$
\begin{array}{lcl}
\textit{last:}\quad \textit{list} & \to & \textit{pos} \\
\textit{last}(l) & = & \begin{cases} p_n & \text{falls } n > 0 \\ \text{undefiniert} & \text{sonst} \end{cases}
\end{array}
$$

$$
\begin{array}{lcl}
\textit{next:}\quad \textit{list} \times \textit{pos} & \to & \textit{pos} \cup \textit{null} \\
\textit{next}(l,p) & = & \begin{cases} p_{i+1} & \text{falls } \exists i \in \{0,\ldots,n-1\} : p = p_i \\ \textit{null} & \text{sonst} \end{cases}
\end{array}
$$

$$
\begin{array}{lcl}
\textit{previous:}\quad \textit{list} \times \textit{pos} & \to & \textit{pos} \cup \textit{null} \\
\textit{previous}(l,p) & = & \begin{cases} p_{i-1} & \text{falls } \exists i \in \{1,\ldots,n\} : p = p_i \\ \textit{null} & \text{sonst} \end{cases}
\end{array}
$$

$$
\begin{array}{lcl}
\textit{bol:}\quad \textit{list} \times \textit{pos} & \to & \textit{bool} \\
\textit{bol}(l,p) & = & (p = p_0)
\end{array}
$$

$$
\begin{array}{lcl}
\textit{eol:}\quad \textit{list} \times \textit{pos} & \to & \textit{bool} \\
\textit{eol}(l,p) & = & (p = p_n)
\end{array}
$$

$$
\begin{array}{lcl}
\textit{insert:} & & \\
\quad \textit{list} \times \textit{pos} \times \textit{elemSet} & \to & \textit{list} \\
\textit{insert}(l,p_i,x) & = & (< a_1,\ldots,a_i,x,a_{i+1},\ldots,a_n >, \\
 & & < p_0,\ldots,p_i,p',p_{i+1},\ldots,p_n >) \\
 & & p' \in \textit{pos} \setminus \{p_0,\ldots,p_n\}
\end{array}
$$

$$
\begin{array}{lcl}
\textit{delete:}\quad \textit{list} \times \textit{pos} & \to & \textit{list} \\
\textit{delete}(l,p) & = & (< a_1,\ldots,a_{i-1},a_{i+1},\ldots,a_n >, \\
 & & < p_0,\ldots,p_{i-1},p_{i+1},\ldots,p_n >)
\end{array}
$$

$$
\begin{array}{lcl}
\textit{concat:}\quad \textit{list} \times \textit{list} & \to & \textit{list} \\
\textit{concat}(& & \\
(< a_1,\ldots,a_n >, < p_0,\ldots,p_n >), & & \\
(< b_1,\ldots,b_m >, < q_0,\ldots,q_m >)) & = & (< a_1,\ldots,a_n,b_1,\ldots,b_m >, \\
 & & < p_0,\ldots,p_n,q_1,\ldots,q_m >)
\end{array}
$$

© R.-R. Grigat 2003-2019

$$
\begin{array}{lll}
\textit{isempty: } \textit{list} & \rightarrow & \textit{bool} \\
\textit{isempty}(l) & = & (n = 0) \\
\\
\textit{find: } \textit{list} \times (\textit{elemSet} \rightarrow \textit{bool}) & \rightarrow & \textit{pos} \cup \textit{null} \\
\textit{find}(l, f) & = & \begin{cases} p_i & \text{falls } \exists i : f(a_i) = \text{true} \\ & & \text{und} \\ & & \forall\, j \in \{1, \ldots, i-1\} : \\ & & f(a_j) = \text{false} \\ \textit{null} & & \text{sonst} \end{cases} \\
\textit{retrieve: } \textit{list} \times \textit{pos} & \rightarrow & \textit{elemSet} \\
\textit{retrieve}(l, p) & = & a_i \text{ falls } \exists i : p_i = p
\end{array}
$$

0.10.1.4 Datenstruktur: Implementierung als doppelt verkettete Liste

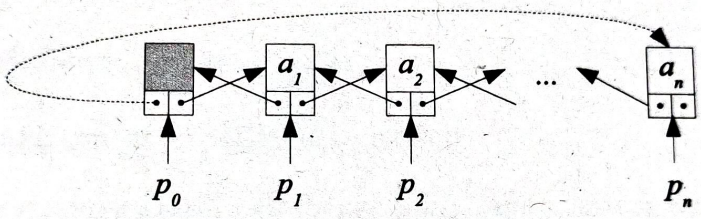


Abbildung 53: Datenstruktur: Implementierung des Datentyps Liste mit Position als Datenstruktur doppelt verkettete Liste

- p_0: Zeiger auf Listenkopf
- Vorgängerzeiger von Listenkopf
  - eigentlich nicht benötigt
  - ermöglichen aber effiziente Operationen *last* und *concat*

Abbildung 54: Leere Liste

Beispiel: KP34_7_weakPtr1

© R.-R. Grigat 2003-2019

0.10.1.5 Einfügen in doppelt verkettete Liste

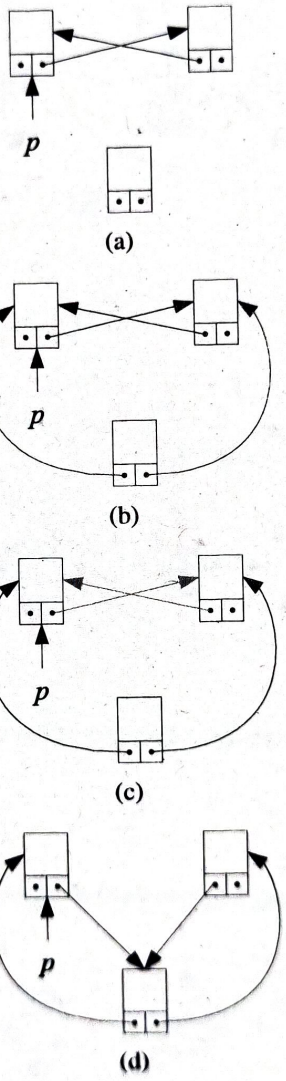
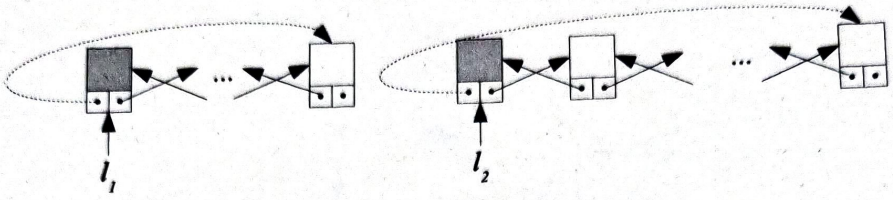


Abbildung 55: Einfügen in doppelt verkettete Liste

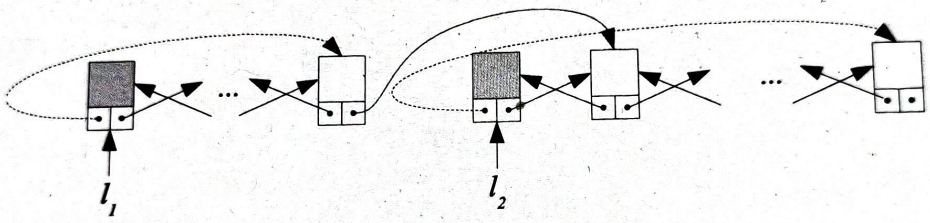
Komplexität: $O(1)$

© R.-R. Grigat 2003-2019

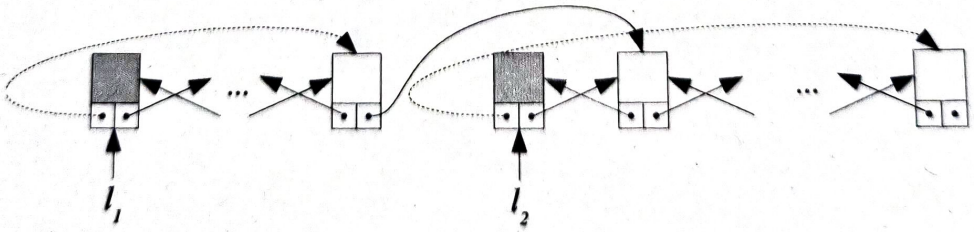
0.10.1.6 Verketten zweier doppelt verketteter Listen



(a)



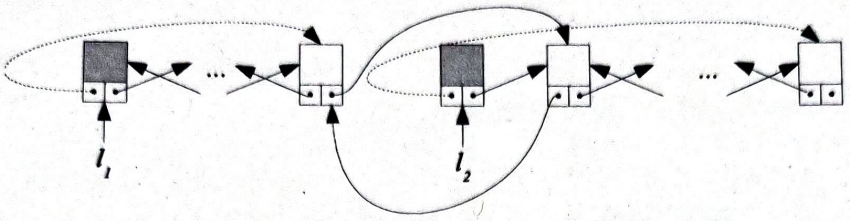
(b)



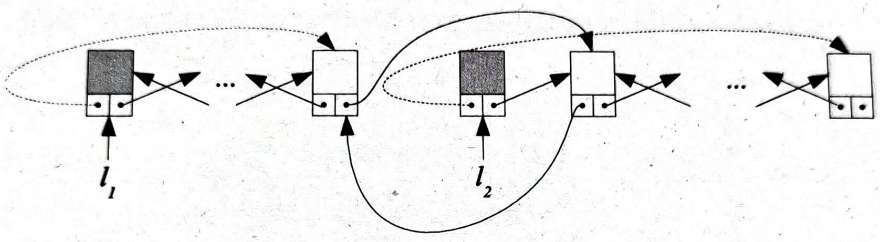
(c)

Abbildung 56: Verketten zweier doppelt verketteter Listen

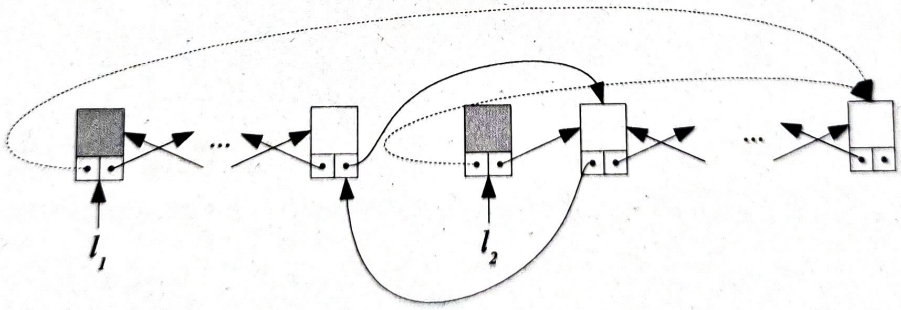
© R.-R. Grigat 2003-2019



(d)



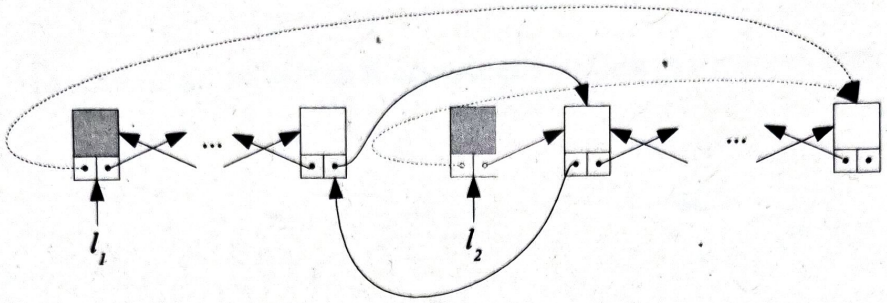
(e)



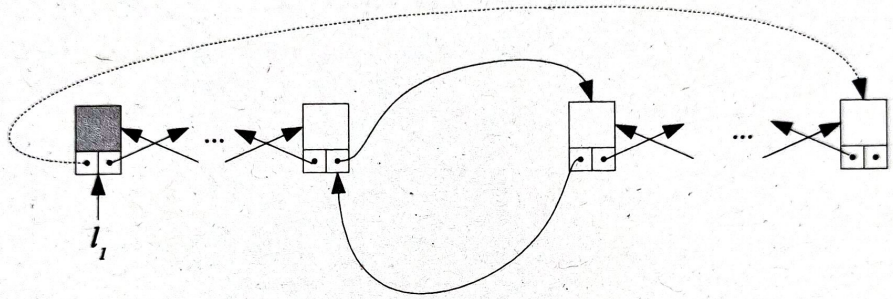
(f)

Abbildung 56: Verketten zweier doppelt verketteter Listen (cont.)

© R.-R. Grigat 2003-2019



(g)



(h)

Abbildung 56: Verketteten zweier doppelt verketteter Listen (cont.)

Komplexität: $O(1)$

0.10.1.7 Datenstruktur: Liste als einfach verkettete Liste (erste Idee)

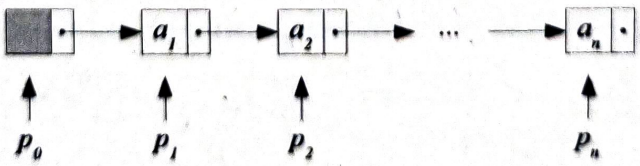


Abbildung 57: Einfach verkettete Liste (erste Idee)

© R.-R. Grigat 2003-2019

Vorteil:

- Weniger Speicherplatz als bei doppelt verketteter Liste notwendig
  - für jedes Listenelement nur einen Zeiger auf Nachfolger

Nachteil:

- Keine effiziente Realisierung der Operationen *previous*, *last*, *concat*
- *delete* nicht möglich
  - Zeiger auf Vorgänger fehlt

0.10.1.8 Datenstruktur: Implementierung einer einfach verketteten Liste

Lösung:

- Zeiger p_i zeigt auf Vorgänger a_{i-1}, p_0 zeigt auf a_n

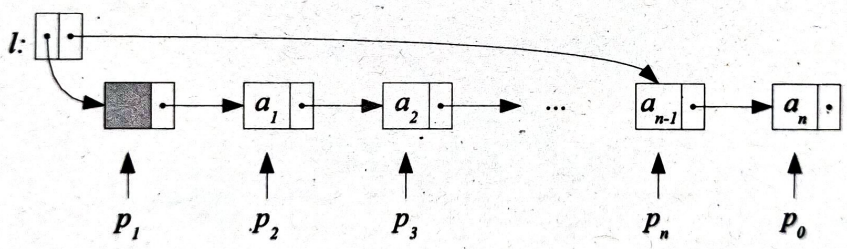


Abbildung 58: Einfach verkettete Liste (optimiert)

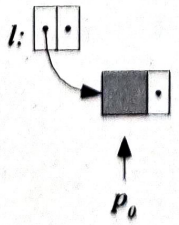


Abbildung 59: Leere einfach verkettete Liste (optimiert)

© R.-R. Grigat 2003-2019

0.10.1.9 Löschen des Elements aus einelementigen einfach verketteten Listen

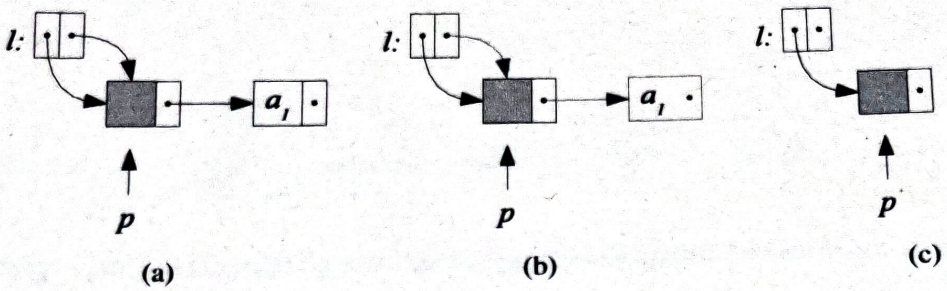
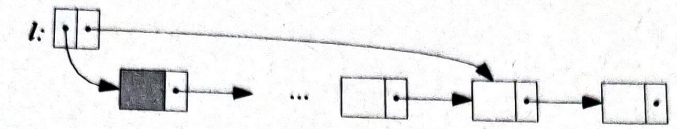


Abbildung 60: Löschen des Elements einer einelementigen einfach verketteten Liste

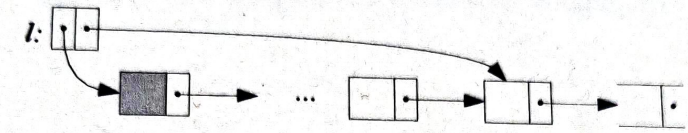
Komplexität: $O(1)$

© R.-R. Grigat 2003-2019

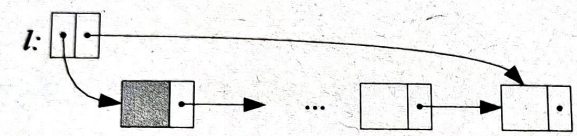
0.10.1.10 Löschen des letzten Elements aus einfach verketteten Listen



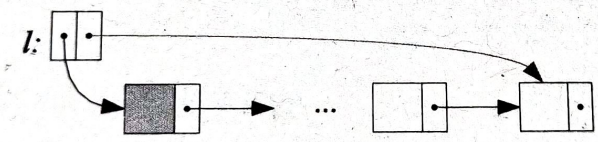
(a)



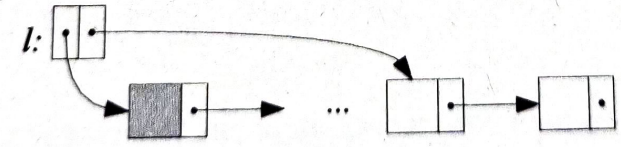
(b)



(c)



(d)



(e)

Abbildung 61: Löschen des letzten Elements einer einfach verketteten Liste

© R.-R. Grigat 2003-2019

Löschen des letzten (oder vorletzten) Elements hat Komplexität: $O(n)$

0.10.1.11 Löschen eines mittleren Elements aus einfach verketteten Listen

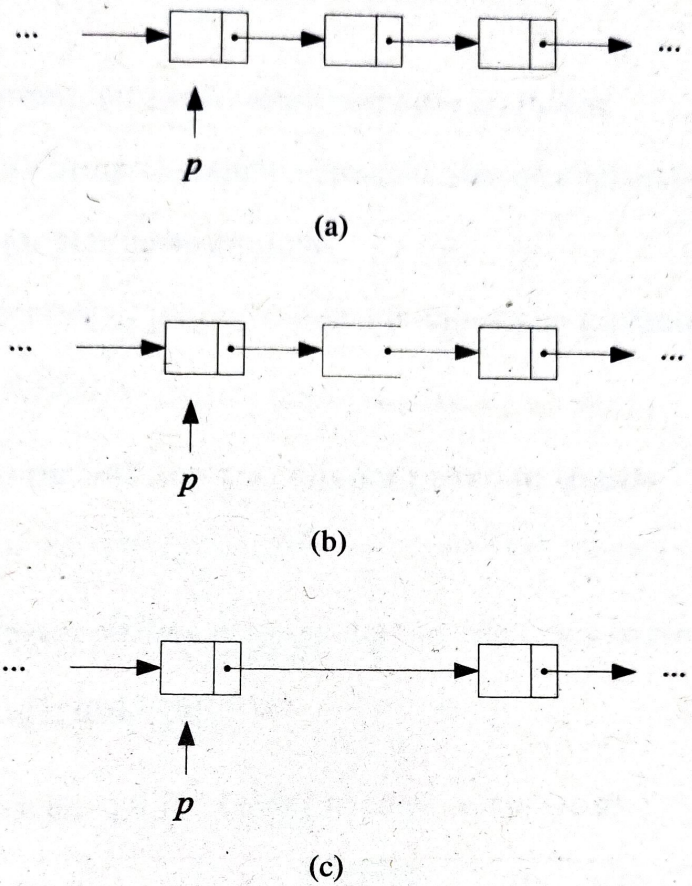


Abbildung 62: Löschen eines mittleren Elements einer einfach verketteten Liste

Komplexität: $O(1)$ Beispiel: KP21_4_einfachVerketteteListe (ohne Einfügen in Mitte)

0.10.1.12 Datenstruktur: Implementierung einer Liste als Array

a_1	a_2	a_3	...	a_n	...

Tabelle 10: Speicherung einer Liste im Array

Nachteile:

- Einfügen und Entfernen aufwendig

© R.-R. Grigat 2003-2019

- nachfolgende Elemente müssen verschoben werden
- $T_{\text{worst}} \in O(n)$

- Maximalgröße der Liste muss vorher festgelegt werden
  - Speicherplatzverschwendung
  - Kompletter Array muss umkopiert werden, falls durch Einfügen Überlauf erzeugt würde

Vorteile:

- Sehr einfache Implementierung
- Keine Notwendigkeit von Zeigern

0.10.1.13 Einfach oder doppelt verkettete Liste als Array

Beispiel

Abbildung 63: Beispielliste

	Index	Wert	Nachfolger
	1		
	2	13	5
L →	3		2
	4	35	0
	5	27	4

Tabelle 11: Beispielliste im Array

© R.-R. Grigat 2003-2019

0.10.1.14 Zwei Listen in einem Array

Beispiel

	Index	Wert	Nachfolger
frei →	1		5
	2	13	7
L →	3		2
	4	35	0
	5		8
M →	6		9
	7	27	4
	8		0
	9	63	0

Tabelle 12: Zwei Listen in einem Array

- Zwei Listen (L, M) in einem Array
- Freie Speicherplätze werden in zusätzlicher Liste (frei) verwaltet

0.10.1.15 Implementierung von verketteten Listen in Arrays

Vorteil:

- Speicheranforderung an Betriebsystem nur einmal zu Programmbeginn
  - Schnelle Programmausführung
- Komplette Array-Struktur kann in externem Speicher abgelegt werden
  - Nach Neustart des Programms weiterhin verfügbar

Nachteil:

- Festlegung des Gesamtspeicherplatzbedarfs vor Programmausführung
  - Beschränkung der Listengröße, falls Array zu klein
  - Speicherplatzverschwendung falls Array zu groß

© R.-R. Grigat 2003-2019

0.10.1.16 Zusammenfassung: Rechenzeit von Listenoperationen

Doppelt verketteten Listen

- Rechenzeit für *find*: $O(n)$
- Rechenzeit für *alle* anderen Listenoperationen: $O(1)$

Einfach verkettete (optimierte) Listen *ohne* extra Zeiger auf vorletztes Element

- Rechenzeit für *find*, *last*, *concat*, *previous*: $O(n)$
- Rechenzeit für *delete*: $O(1)$
- Rechenzeit für alle anderen Listenoperationen: $O(1)$

Einfach verkettete (optimierte) Listen *mit* extra Zeiger auf vorletztes Element

- Rechenzeit für *last*, *concat*: $O(1)$
- Rechenzeit für *find*, *previous*, *delete*(p_n): $O(n)$
- Rechenzeit für *delete* sonst: $O(1)$
- Rechenzeit für alle anderen Listenoperationen: $O(1)$

0.10.2 Stacks (Stapel)

0.10.2.1 Stacks (Stapel)

Stacks (Stapel)

- Spezialfall von Listen
- LIFO (last-in-first-out)
- Entspricht Modell einer einfache Liste ohne *concat*
- Traditionelle Bezeichnungen der Operationen:

stack		*list*
top	$\widehat{=}$	*first*
push	$\widehat{=}$	*append*
pop	$\widehat{=}$	*first* und *rest*

© R.-R. Grigat 2003-2019

0.10.2.2 Datenstruktur: Stack-Implementierung als Array

Stack-Implementierung als Array

- Einfügen und Entfernen der Elemente nur an einem Ende der Liste
- Ineffizientes Verschieben der Folgeelemente entfällt

	Index	Element
	5	
	4	
$\rightarrow$ *top*	3	a_1
	2	a_2
	1	a_3

Tabelle 13: Stack-Implementierung als Array

Beispiel: KP32_1_stackTemplateT, KP32_1_stackTemplateTn

0.10.3 Queues (Warteschlangen)

0.10.3.1 Queues (Warteschlangen)

Queues (Warteschlangen)

- Spezialfall von Listen
  - Elemente werden nur an einem Ende („vorne“) entnommen
  - Elemente werden nur am anderen Ende angehängt („hinten“)
  - FIFO (first-in-first-out)
- Entspricht Modell einer einfache Liste
  - ohne *concat* und *append*
  - aber mit zusätzlicher Listenoperation *rappend* (rear-*append*)

$$\begin{aligned} &\textit{rappend:} \quad \textit{list} \times \textit{elemSet} \quad \rightarrow \quad \textit{list} \\ &\textit{rappend}(< a_1, \ldots, a_n >, e) \quad = \quad < a_1, \ldots, a_n, e > \end{aligned}$$

- Datentyp *queue* entspricht dem Datentyp *Liste*

© R.-R. Grigat 2003-2019

- Traditionelle Bezeichnungen der Operationen:

queue		***list***
front	$\widehat{=}$	*first*
enqueue	$\widehat{=}$	*rappend*
dequeue	$\widehat{=}$	*first* und *rest*

0.10.3.2 Datenstruktur: Queue-Implementierung als Array

		a_1	a_2	a_3	...	a_n			

Tabelle 14: Queue-Implementierung als Array

Problem:

- Queue „wandert“ im Array durch Einfügen und Entfernen an verschiedenen Enden

Lösung:

- Array als Zyklus auffassen

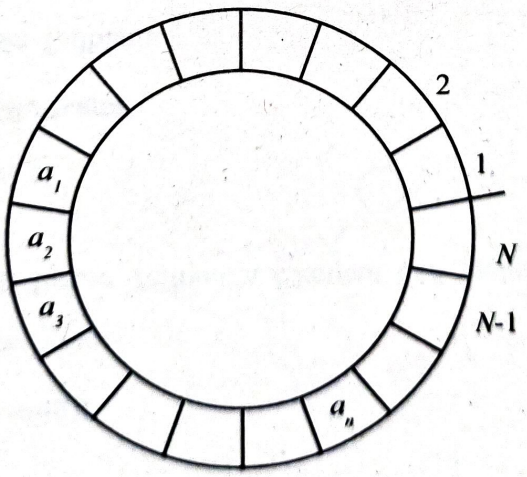


Abbildung 64: Queue-Implementierung als zyklisches Array (Ringbuffer)

© R.-R. Grigat 2003-2019

0.10.4 Binäre Bäume

0.10.4.1 Binäre Bäume: Nomenklatur

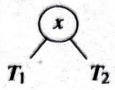


Abbildung 65: Graphische Darstellung des Tripels (T_1, x, T_2)

Definition:

- Der leere Baum ($\diamond$) ist binärer Baum
- x ist Knoten und T_1, T_2 sind binäre Bäume $\Longleftrightarrow$ Tripel (T_1, x, T_2) ist binärer Baum
- Bezeichnungen:
  - *Knoten*: Elemente des Baums
  - *Kante*: Verbindung zweier Knoten
  - *Wurzel*: Knoten x in Baum (T_1, x, T_2)
  - *linker Teilbaum*: T_1
  - *rechter Teilbaum*: T_2
  - *Söhne* von x: Wurzeln der Teilbäume T_1, T_2
  - *Vater* der Wurzeln der Teilbäume T_1, T_2 ist x
  - *Blatt*: Knoten, der keine Söhne hat
  - *Innerer Knoten*: Knoten, der nicht Blatt ist
  - *Schlüsselwert*: Einer der in Element (d.h. Knoten) gespeicherten Werte, dient z. B. zum Sortieren oder Suchen von Elementen

© R.-R. Grigat 2003-2019

0.10.4.2 Binäre Bäume: Definition

Datentyp: binärer Baum

Funktionen (Beschreibung):

- *new* erzeugt leeren Baum
- *maketree* verknüft linken Teilbaum, Element und rechten Teilbaum zu neuem Baum
- *root* liefert Wurzel
- *left* liefert linken Teilbaum
- *right* liefert rechten Teilbaum
- *isempty* prüft, ob Baum leer ist
- *key* liefert Schlüsselwert eines Knotens bzw. Elements

Datentypen

$$\begin{array}{lll} bool & & \\ elemSet & & \\ tree & = & \{\diamond\} \cup \{(L,x,R) | x \in elemSet,\ L,R \in tree\} \end{array}$$

© R.-R. Grigat 2003-2019

Funktionen (Definition):
Sei $T = (L,x,R)$.

$$
\begin{array}{lll}
\textit{new:} & \rightarrow & \textit{tree} \\
\textit{new} & = & \diamond \\
 & & \\
\textit{maketree:} \quad \textit{tree} \times \textit{elemSet} \times \textit{tree} & \rightarrow & \textit{tree} \\
\textit{maketree}(L,x,R) & = & (L,x,R) \\
 & & \\
\textit{root:} \quad \textit{tree} & \rightarrow & \textit{elemSet} \\
\textit{root}(T) & = & \begin{cases} x & \text{falls } T = (L,x,R) \\ \text{undefiniert} & \text{sonst} \end{cases} \\
 & & \\
\textit{left:} \quad \textit{tree} & \rightarrow & \textit{tree} \\
\textit{left}(T) & = & \begin{cases} L & \text{falls } T = (L,x,R) \\ \text{undefiniert} & \text{sonst} \end{cases} \\
 & & \\
\textit{right:} \quad \textit{tree} & \rightarrow & \textit{tree} \\
\textit{right}(T) & = & \begin{cases} R & \text{falls } T = (L,x,R) \\ \text{undefiniert} & \text{sonst} \end{cases} \\
 & & \\
\textit{isempty:} \quad \textit{tree} & \rightarrow & \textit{bool} \\
\textit{isempty}(T) & = & \begin{cases} \textit{true} & \text{falls } T = \diamond \\ \textit{false} & \text{sonst} \end{cases} \\
 & & \\
\textit{key:} \quad \textit{elemSet} & \rightarrow & \textit{number} \\
\textit{key}(x) & = & \begin{cases} x & \text{falls } x \text{ Schlüsselwert} \\ k & \text{falls } x \text{ Tupel } (\ldots,k,\ldots) \\ & \text{und } k \text{ Schlüsselwert} \\ \text{undef.} & \text{sonst} \end{cases}
\end{array}
$$

0.10.4.3 Pfade

Definition: Pfad

- Folge von Knoten $< p_0, \ldots, p_n >$, so dass p_i mit $i \in \{0, \ldots, n-1\}$ jeweils Vater von p_{i+1} ist

Definition: Länge laenge eines Pfades

- $\text{laenge}(< p_0, \ldots, p_n >) = n$
- Anzahl der Kanten auf dem Pfad

Definition: Höhe h eines Baumes T

- Länge des längsten Pfades $\text{laenge}_{\text{max}}$
- Führt von der Wurzel zu einem Blatt

Tiefe $\text{tiefe}_T(p)$ eines Knotens p in Baum T

- $\text{tiefe}_T(p) = \text{laenge}(< p_{\text{Wurzel}}, \ldots, p >)$

0.10.4.4 Maximale Höhe

Maximale Höhe eines Binärbaumes mit n Knoten:

- $h_{\text{max}}(n) = n - 1 \in O(n)$
- zu Liste entarteter Baum

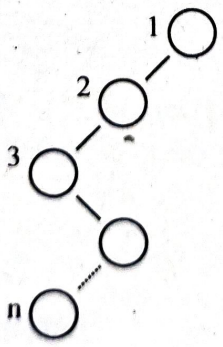


Abbildung 66: Zur Liste entarteter Baum

© R.-R. Grigat 2003-2019

0.10.4.5 Minimale Höhe

Definition: vollständiger Baum

- Ein Baum heißt *vollständig*, wenn alle Ebenen, ggf. bis auf die unterste nicht-leere Ebene, vollständig besetzt sind.
- Ein vollständiger Baum heißt *links-vollständig*, wenn die unterste nicht-leere Ebene von links her bis zu einem beliebigen Blatt ohne Lücken besetzt und rechts davon leer ist.

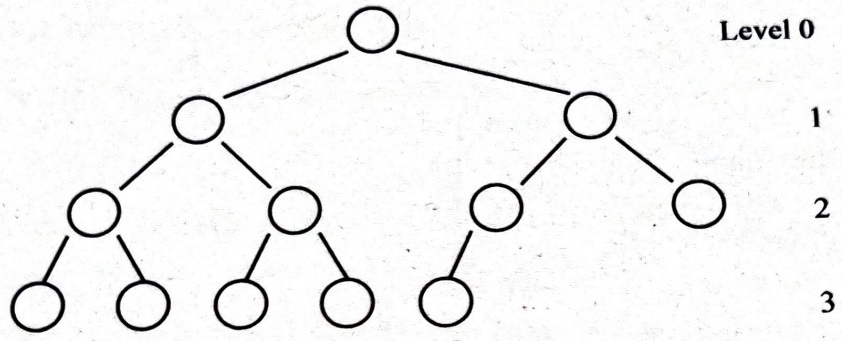


Abbildung 67: Vollständiger Baum

Minimale Höhe eines Binärbaumes mit n Knoten:
Die maximale Knotenzahl N_h eines Binärbaumes der Höhe h beträgt

$$N_h = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1$$

Ein vollständiger Baum hat minimale Höhe. Deshalb gilt:

$$\begin{array}{rcccl} N_{h-1}+1 & \leq & n & < & N_h+1 \\ (2^h-1)+1 & \leq & n & < & (2^{h+1}-1)+1 \\ h & \leq & \log_2 n & < & h+1 \\ h_{\min} & := & h & = & \lfloor \log_2 n \rfloor \end{array}$$

wobei $\lfloor x \rfloor$ mit $x \in \mathbb{R}$ den nächstkleineren Ganzzahlwert $z \in \mathbb{Z}$ mit $z \leq x < z+1$ bedeutet.

© R.-R. Grigat 2003-2019

0.10.4.6 Abbilden eines Baumes auf eine Liste

Alle Elemente eines Baumes besuchen:

- *inorder*

$$\begin{array}{lcl} inorder: \ tree & \rightarrow & list \\ inorder(\diamond) & = & \diamond \\ inorder\big((L,x,R)\big) & = & inorder(L) \circ < x > \circ inorder(R) \end{array}$$

- *preorder*

$$\begin{array}{lcl} preorder: \ tree & \rightarrow & list \\ preorder(\diamond) & = & \diamond \\ preorder\big((L,x,R)\big) & = & < x > \circ preorder(L) \circ preorder(R) \end{array}$$

- *postorder*

$$\begin{array}{lcl} postorder: \ tree & \rightarrow & list \\ postorder(\diamond) & = & \diamond \\ postorder\big((L,x,R)\big) & = & postorder(L) \circ postorder(R) \circ < x > \end{array}$$

Beispiel

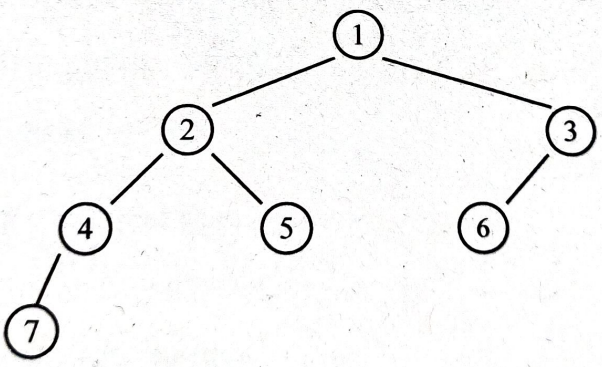


Abbildung 68: Beispielbaum T

$$\begin{array}{lcl} inorder(T) & = & <7,4,2,5,1,6,3> \\ preorder(T) & = & <1,2,4,7,5,3,6> \\ postorder(T) & = & <7,4,5,2,6,3,1> \end{array}$$

© R.-R. Grigat 2003-2019

0.10.4.7 Operatorbaum

Operatorbaum

- Darstellung eines arithmetischen Ausdrucks als binärer Baum

Beispiel

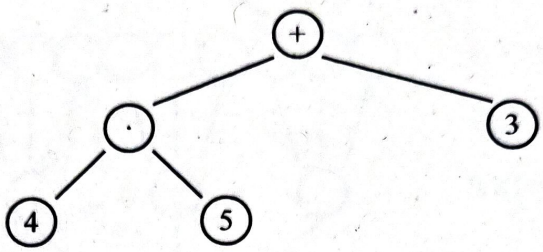


Abbildung 69: Operatorbaum T

$$
\begin{array}{lcll}
inorder(T) & = & <4,\cdot,5,+,3> & \text{(Infix-Notation)} \\
preorder(T) & = & <+,\cdot,4,5,3> & \text{(Prefix-Notation)} \\
postorder(T) & = & <4,5,\cdot,3,+> & \text{(Postfix-Notation)}
\end{array}
$$

0.10.4.8 Datenstruktur: Implementierung von Binärbäumen mit Zeigern

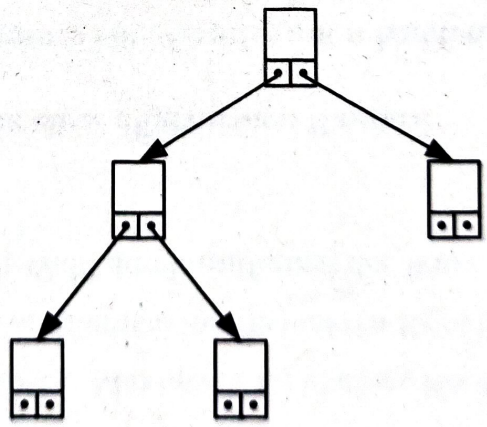


Abbildung 70: Implementierung eines Binärbaumes mit Zeigern

© R.-R. Grigat 2003-2019

0.10.4.9 Datenstruktur: Implementierung von Binärbäumen mit Arrays

Einbettung eines Binärbaumes in Array

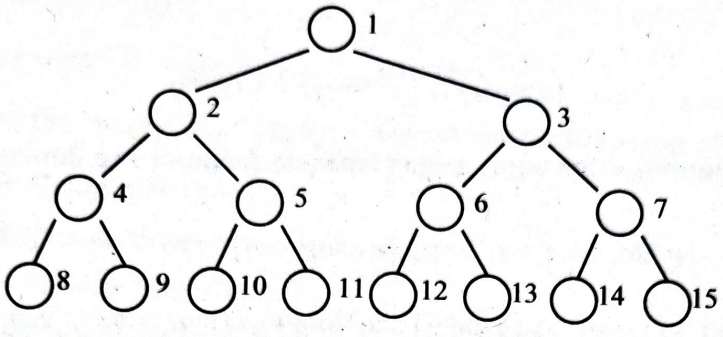


Abbildung 71: Vollständiger binärer Baum der Höhe h

- Baum der Höhe h kann in Array der Länge $2^{h+1}-1$ eingebettet werden
- Für jeden Knoten p gilt:

$$\begin{aligned} .Index(p) = i \quad \Rightarrow \quad Index(linker\ Sohn(p)) &= 2 \cdot i \\ Index(rechter\ Sohn(p)) &= 2 \cdot i + 1 \end{aligned}$$

0.10.5 Allgemeine Bäume

0.10.5.1 Allgemeine Bäume: Definition und Nomenklatur

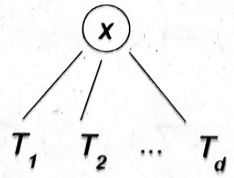


Abbildung 72: Graphische Darstellung des $(d+1)$-Tupels $(x, T_1, \ldots, T_d)$

Definition:

- Ein einzelner Knoten x ist ein Baum.
- x ist ein Knoten, $T_1, \ldots, T_d$ sind Bäume $\Rightarrow$ $(d+1)$-Tupel $(x, T_1, \ldots, T_d)$ ist Baum
- Bezeichnungen:

© R.-R. Grigat 2003-2019

- *Wurzel*: x
- *Söhne* von x sind die Wurzeln von $T_1, \ldots, T_d$
- *Vater* der Wurzeln von $T_1, \ldots, T_d$ ist x
- *Grad eines Knotens*: Anzahl seiner Söhne
- *Grad eines Baumes*: Maximaler Grad eines Knotens im Baum
- *Wald*: Menge von Bäumen mit disjunkten Knotenmengen
  - Man erhält Wald durch Entfernen der Wurzel eines Baumes

0.10.5.2 Maximale Höhe eines allgemeinen Baumes

Maximale Höhe eines Baumes vom Grad d mit n Knoten:

- $h_{\max}(n) = n - 1 \in O(n)$
- zu einer Liste entarteter Baum (siehe Abb. 66)

0.10.5.3 Maximale Knotenzahl und minimale Höhe

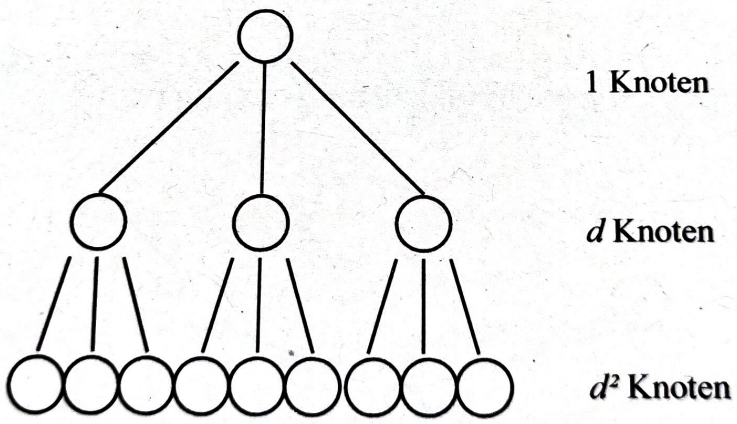


Abbildung 73: Vollständiger Baum vom Grad $d = 3$

Maximale Knotenzahl N_h eines Baumes vom Grad d mit Höhe h

$$N_h = \sum_{i=0}^{h} d^i = ?$$

© R.-R. Grigat 2003-2019

Satz: Maximale Knotenzahl N_h für Grad d und Höhe h

$$N_h = \sum_{i=0}^{h} d^i = \frac{d^{h+1}-1}{d-1} \quad \forall d \neq 1$$

Beweis:

Es gilt $N_h = \sum_{i=0}^{h} d^i$.

$$N_{h+1} = d^{h+1} + N_h = d^0 + \sum_{i=1}^{h+1} d^i = 1 + \sum_{\hat{i}=0}^{h} d^{\hat{i}+1} = 1 + d \cdot \sum_{\hat{i}=0}^{h} d^{\hat{i}} = 1 + d \cdot N_h$$

$$\begin{array}{lrcl} & d^{h+1} + N_h & = & 1 + d \cdot N_h \\ \Leftrightarrow & d^{h+1} - 1 & = & (d-1) \cdot N_h \\ \Leftrightarrow & N_h & = & \frac{d^{h+1}-1}{d-1} \quad \forall d \neq 1 \end{array}$$

Minimal erforderliche Höhe eines Baumes vom Grad d mit n Knoten (und $N_h = \frac{d^{h+1}-1}{d-1}$)

Ein *vollständiger* Baum bei gegebener Knotenzahl hat minimale Höhe und es gilt:

$$\begin{array}{lcll} N_{h-1} & < & n & \leq N_h \\ \frac{d^h-1}{d-1} & < & n & \leq \frac{d^{h+1}-1}{d-1} \\ d^h & < & n \cdot (d-1) + 1 & \leq d^{h+1} \\ h & < & \log_d(n \cdot (d-1)+1) & \leq h+1 \\ h & = & \lceil \log_d(n \cdot (d-1)+1) \rceil - 1 & \end{array}$$

Obere Abschätzung der minimal erforderlichen Höhe: Mit $n \cdot (d-1)+1 = n \cdot d - n + 1 \leq n \cdot d$ folgt

$$h \leq \lceil \log_d nd \rceil - 1 = \lceil \log_d n \rceil + \log_d d - 1 = \lceil \log_d n \rceil$$

Obere Abschätzung der minimal erforderlichen Höhe eines Baumes vom Grad d mit n Knoten:

$$h_{\min}(n) \in O(\log_d n) = O(\log n)$$

© R.-R. Grigat 2003-2019

0.10.5.4 Implementierung von allgemeinem Baum als Binärbaum

Abbildung zwischen allg. Baum und Binärbaum

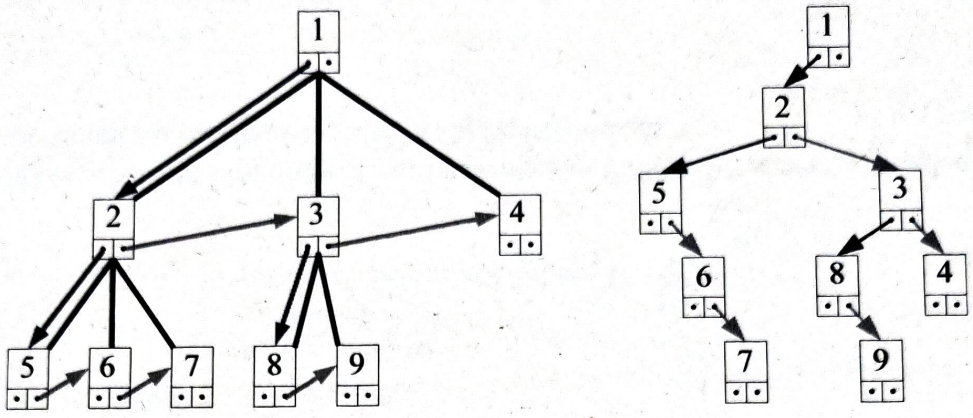


Abbildung 74: Implementierung des allg. Baumes (Linien) als Binärbaum (Pfeile)

- Allg. Baum: Sohn $p_{l,\text{allg}}$ ganz links bzgl. Vater $p_{V,\text{allg}} \Longrightarrow$ Binärbaum: ***linker*** Sohn $p_{l,\text{bin}} \equiv p_{l,\text{allg}}$ des Vaters $p_{V,\text{bin}} = p_{V,\text{allg}}$
- Allg. Baum: Bruder $p_{i+1,\text{allg}}$ rechts von Sohn $p_{i,\text{allg}} \Longrightarrow$ Binärbaum: ***rechter*** Sohn $p_{r,\text{bin}} \equiv p_{i+1,\text{allg}}$ des Vaters $p_{V,\text{bin}} \equiv p_{i,\text{allg}}$

© R.-R. Grigat 2003-2019

0.11 Mengen, Dictionaries und Priority Queues

0.11.1 Operationen für Mengen

0.11.1.1 Datentyp: Mengen

- Bisher diskutierte Datentypen: Listen, Bäume
- Motivation für Mengen als Datentyp: 8 (mathematische) Mengenfunktionen
  - Neue Menge erzeugen
  - Einfügen von Elementen
  - Entfernen von Elementen
  - Enthaltensein von Elementen
  - Vereinigung zweier Mengen
  - Durchschnitt zweier Mengen
  - Differenz zweier Mengen
  - Ausgabe der Menge als Liste

  sollen für Mengentypen effizient durch Algorithmen implementiert werden können.

0.11.1.2 Modell einer Menge mit Durchschnitt, Vereinigung, Differenz

Datentyp: Menge

Funktionen (Beschreibung):

- *new* liefert eine leere Menge
- *insert* fügt ein Element in die Menge ein
- *delete* löscht ein Element der Menge
- *member* prüft, ob ein Element enthalten ist
- *union* bildet aus zwei Mengen die Vereinigung
- *intersection* bildet aus zwei Mengen die Schnittmenge
- *difference* bildet aus zwei Mengen die Differenzmenge

© R.-R. Grigat 2003-2019

- *enumerate* liefert eine Kopie aller Elemente als Liste

Funktionen (Signatur des Datentyps):

new:		$\rightarrow$	*set*
insert:	*set* $\times$ *element*	$\rightarrow$	*set*
delete:	*set* $\times$ *element*	$\rightarrow$	*set*
member:	*set* $\times$ *element*	$\rightarrow$	*bool*
union:	*set* $\times$ *set*	$\rightarrow$	*set*
intersection:	*set* $\times$ *set*	$\rightarrow$	*set*
difference:	*set* $\times$ *set*	$\rightarrow$	*set*
enumerate:	*set*	$\rightarrow$	*list*

0.11.1.3 Datenstruktur: Implementierung einer Menge als Bitvektor

Bitvektor b* der Länge *N

- kann Menge S mit maximal N Elementen darstellen
- `b[i] == true` $\Longleftrightarrow e_i \in S$
- Komplexität der Mengenoperationen:

new	$O(N)$
insert	$O(1)$
delete, member	$O(1)$
enumerate	$O(N)$
union, intersection, difference	$O(N)$

- Aufwand nicht proportional zur tatsächlichen Größe der Menge n, sondern proportional zur *maximal möglichen* Größe der Menge N

0.11.1.4 Datenstruktur: Implementierung einer Menge als ungeordnete Liste

Datenstruktur: Menge als *ungeordnete* Liste

- *Liste*, die die Elemente der Menge *unsortiert* enthält

© R.-R. Grigat 2003-2019

- Komplexität der Mengenoperationen bei zwei Mengen der Größen m und n (ohne Duplikate):

Menge implementiert als	Bit-vektor	*ungeordnete Liste*
new	$O(N)$	$O(1)$
insert (ohne Duplikate)	$O(1)$	$O(n)$
delete, member	$O(1)$	$O(n)$
enumerate	$O(N)$	$O(n)$
union, intersection, difference	$O(N)$	$O(m \cdot n)$

0.11.1.5 Datenstruktur: Implementierung einer Menge als geordnete Liste

Datenstruktur: Menge als *geordnete* Liste

- *Liste*, die die Elemente der Menge *sortiert* enthält
- Komplexität der Mengenoperationen bei zwei Mengen der Größen m und n (ohne Duplikate):

Menge implementiert als	Bit-vektor	ungeord. Liste	*geordnete Liste*
new	$O(N)$	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$	$O(n)$
delete, member	$O(1)$	$O(n)$	$O(n)$
enumerate	$O(N)$	$O(n)$	$O(n)$
union, intersection und *difference*	$O(N)$	$O(m \cdot n)$	$O(m+n)$

0.11.1.6 Vorteil einer geordneten Liste

Operationen *union, intersection, difference*

- Operationen *union, intersection, difference* benötigen nur $O(m+n)$ Schritte
- Realisierung durch parallelen Durchlauf durch beide Listen
  - Elemente an aktueller Position beider Listen werden verglichen

© R.-R. Grigat 2003-2019

– *next* wird ausgeführt für Liste mit kleinerem Element

Beispiel:

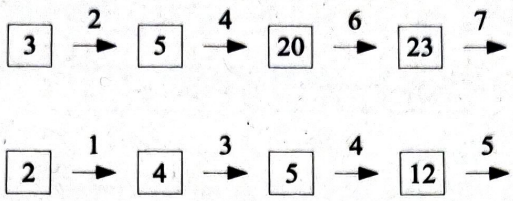


Abbildung 75: Reihenfolge beim parallelen Durchlaufen zweier Listen

0.11.2 Dictionaries (Wörterbücher)

Datentyp: Dictionary (Wörterbuch)

- Datentyp, enthält im Gegensatz zu Mengen nur die drei (mathematischen) Funktionen
  - *insert*
  - *delete*
  - *member* (prüft, ob Element in Menge enthalten ist)
- Anforderungen an eine schnelle Implementierung:
  1. Zeitkomplexität $O(1)$ für *insert, delete, member*
     - Liste ungeeignet
  2. Speicherplatzbedarf $O(n)$, linear abhängig von Mächtigkeit n der Menge
     - Bitvektor ungeeignet
     - Bitvektoren für maximale Mächtigkeit ausgelegt

© R.-R. Grigat 2003-2019

0.11.2.1 Datenstruktur: Implementierungen von Dictionaries

Ziel:

- Rechenzeit $O(1)$ für *insert, delete, member*
- Speicherbedarf möglichst $O(n)$

Lösungen für Datenstruktur Dictionary:

1. Hashing
2. Binäre Suchbäume
3. AVL-Bäume

0.11.3 Hashing

0.11.3.1 Implementierung von Dictionaries: Hashing

Grundidee

- Zusammenfassen von Elementen zu Behältern (*Buckets*) in einer Tabelle (Hash-Tabelle)
- Aus dem Schlüsselwert bzw. *Schlüssel* eines Elements den dazugehörigen Behälter *berechnen*
- Innerhalb des Behälters „ineffizientere" Methoden verwenden, z. B. die Suche in Listen

Beispiel: Abholung von Fotoarbeiten in einer Drogerie

- Element: Fotoversandtasche
- Schlüssel: Kennnummer der Versandtasche
- Behälter: Fächer, in die Fototaschen sortiert werden
- Berechnungsvorschrift für die Behälter: Endziffer der Kennnummer

© R.-R. Grigat 2003-2019

0.11.3.2 Hashfunktion

- Hashing verwenden, wenn $|D| \gg m$
  - D: Wertebereich der zu speichernden Schlüssel k
  - m: Anzahl der verfügbaren Behälter
- Hashfunktion $h : D \to \{0, \ldots, m-1\}$
- Anforderungen an eine Hashfunktion:
  - *Surjektivität*, d. h. alle Behälter erfassend.
  - Schlüssel sollen *gleichmäßig* über alle Behälter verteilt werden
  - *Effiziente* Berechnung

0.11.3.3 Hashfunktionen: Divisionsmethode

Divisionsmethode

$$h(k) = k \mod m \tag{1}$$

- Nachteil: aufeinanderfolgende Schlüsselwerte k werden auf aufeinanderfolgende Zellen abgebildet

0.11.3.4 Hashfunktionen: Mittel-Quadrat-Methode

Mittel-Quadrat-Methode

$$h(k) = \text{mittlere Ziffern von } k^2 \tag{2}$$

Beispiel: $m = 100$

k	$k \mod m$	k^2	$h(k)$
111	11	12321	32
112	12	12544	54
113	13	12769	76

Tabelle 15: Beispiel zu Hashfunktionen

- gute Streuung, denn $h(k)$ hängt von allen Ziffern in k ab

© R.-R. Grigat 2003-2019

0.11.3.5 Nomenklatur

Bezeichnungen

- *Offenes Hashing*: Jeder Behälter darf beliebig viele Schlüssel enthalten
- *Geschlossenes Hashing*: Maximale Anzahl b_{max} an Schlüsseln b pro Behälter wird vorgegeben (im Folgenden wird nur $b_{max} = 1$ betrachtet)
- *Offene Adressierung*: Elemente werden in Hashtabelle direkt gespeichert (impliziert geschlossenes Hashing mit $b_{max} = 1$)
- *Kollision*: Mehrere Schlüssel werden auf denselben Behälter abgebildet
- *Überlauf*: Bei geschlossenem Hashing existieren für einen Behälter mindestens b_{max} Kollisionen

0.11.3.6 Beispiel: Offenes Hashing mit Divisionsmethode

Beispiel: Hashing von Wochentagen mit Divisionsmethode

- Hashfunktion: *Anzahl an Buchstaben* mod 6
- Kollisionen treten bei den Behältern 1 und 2 auf

h(k)	Behälterinhalt
0	Montag
1	Freitag, Sonntag
2	Dienstag, Mittwoch
3	Sonnabend
4	Donnerstag
5	-

Tabelle 16: Hashtabelle zu Beispiel

© R.-R. Grigat 2003-2019

0.11.3.7 Wahrscheinlichkeit von Kollisionen

Kollisionswahrscheinlichkeit $P_{\text{Kollision},n}$

- Annahme: Hashfunktion bildet auf alle Behälter gleichmäßig ab (ideale Hashfunktion)
- P_i: Wahrscheinlichkeit, dass i-ter Schlüssel auf freien Behälter abgebildet wird
- Gesamtwahrscheinlichkeit für Kollision $P_{\text{Kollision},n}$ bei n Schlüsseln:

$$P_{\text{Kollision},n} = 1 - P_{\text{keine Kollision},n} \tag{3}$$

$$P_{\text{keine Kollision},n} = P_1 \cdot P_2 \cdot \ldots \cdot P_n \tag{4}$$

- Wahrscheinlichkeiten der Abbildung auf freien Behälter

$$P_1 = 1 \tag{5}$$

$$P_2 = \frac{m-1}{m} \tag{6}$$

$$P_i = \frac{m-i+1}{m} \tag{7}$$

- Wahrscheinlichkeit für eine Kollision bei n Schlüsseln und m Behältern ($n < m$):

$$P_{\text{Kollision},n} = 1 - \frac{\prod_{i=1}^{n}(m-i+1)}{m^n} \tag{8}$$

0.11.3.8 Beispiel zu Kollisionswahrscheinlichkeiten

Beispiel: $m = 365$

n	$P_{\text{Kollision}}$
22	0.475
23	0.507
50	0.970

Tabelle 17: Kollisionswahrscheinlichkeiten für n Schlüssel in $m = 365$ Behältern

- „Geburtstagsparadoxon“:

© R.-R. Grigat 2003-2019

- Bei 23 Personen ist Wahrscheinlichkeit 50 %, dass zwei Personen am gleichen Tag Geburtstag haben.
- Bei 50 Personen haben fast sicher zwei Personen am gleichen Tag Geburtstag.

- Beispiel zeigt, dass bei Hashing Kollisionen normalerweise unvermeidlich sind

0.11.3.9 Geburtstagsparadoxon

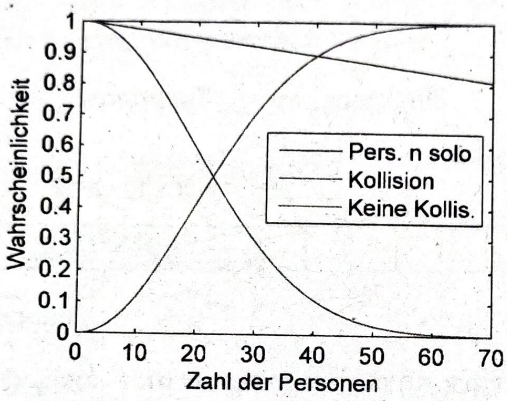


Abbildung 76: Wahrscheinlichkeit, dass die *n*-te Person von *n* Personen allein Geburtstag hat (Person *n* solo), dass bei *n* Personen keine Kollision auftritt und dass mindestens eine Kollision auftritt. Dies entspricht Hashing für 365 Behälter und n Datensätze.

© R.-R. Grigat 2003-2019

0.11.3.10 Offenes Hashing

- Implementierung der Behälter z. B. als beliebig erweiterbare Liste von Schlüsseln
- Zeitkomplexität für *member*, *insert* und *delete*: $O(1+\frac{n}{m})$
  ($\frac{n}{m}$: durchschnittliche Listenlänge)
  - Bester Fall: $O(1)$
    - Anzahl Elemente $n \approx$ Anzahl Behälter m
    - gleichmäßig abbildende Hashfunktion
  - Schlechtester Fall: $O(n)$
    - schlecht abbildende Hashfunktion
      (alle Schlüssel im selben Behälter)
    - nur ein Behälter
- Speicherplatzbedarf: $O(n+m)$

0.11.3.11 Geschlossenes Hashing

- Maximale Anzahl an Elementen: $b_{max} \cdot m$
- Im Folgenden: Spezialfall $b_{max} = 1$, andere Fälle analog handhabbar
- Effiziente Methoden zur Behandlung von Überläufen der Behälter notwendig
- *Rehashing*: Finden eines neuen Behälters im Falle eines Überlaufs über zusätzliche Hashfunktion

0.11.3.12 Rehashing

- Bei Überlauf neben der ursprünglichen Hashfunktion $h = h_0$ weitere Hashfunktionen $h_1, h_2, \ldots$ verwenden, um weitere Zellen der Tabelle zu inspizieren
- *insert*: Anwenden einer Sequenz von Hashfunktionen, bis ein freier Behälter gefunden wird
- *member*, *delete*: Anwenden derselben Sequenz, bis das gesuchte Element gefunden wird.

© R.-R. Grigat 2003-2019

  - Rehashing-Folge darf für Suche von *member*, *delete* nicht durch vorheriges *delete* unterbrochen werden
  - *delete* hinterlässt daher bei Löschung eine Markierung
- Unterschiedliche Kollisionsstrategien:
  - Lineares Sondieren
  - Quadratisches Sondieren
  - Doppeltes Hashing

0.11.3.13 Methoden des Rehashings: Lineares Sondieren

Lineares Sondieren

- Sequenz von Hashfunktionen $h_i(x)$ mit

$$h_i(x) = (h(x) + i) \mod m \tag{9}$$

$\Rightarrow$ Bei einem vollen Behälter wird anschließend zyklisch der darauf folgende getestet

- Nachteil: Kettenbildung

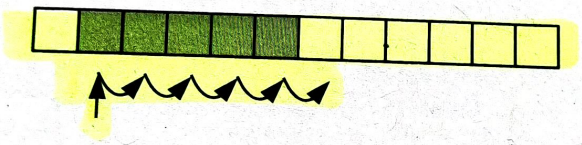


Abbildung 77: Kettenbildung

0.11.3.14 Kettenbildung

Kettenbildung

- Entstehung einer Behältersequenz (Kette) von vollen Behältern, die bei den Operationen *insert*, *delete* und *member* häufig durchlaufen werden muss.
- Konsequenz: Komplexität des offenen Hashings verschlechtert sich von $O(1)$ in Richtung O(Kettenlänge)
- Im Folgenden: Komplexitätsanalyse des „idealen" geschlossenen Hashings als Optimalfall

© R.-R. Grigat 2003-2019

0.11.3.15 Ideales Hashing

- Modell zur Komplexitätsanalyse
- Grundlage: Sequenz von Hashfunktionen, die die Behälter so durchlaufen, dass die Behälterreihenfolge als *zufällig* angenommen werden darf (ohne Kettenbildung)
- Dadurch ist Komplexitätsanalyse mit Wahrscheinlichkeitsbetrachtungen möglich
- Die Komplexität des *idealen* Hashings ist eine untere Schranke für realisierbare Hashingverfahren
- Alle beschriebenen Implementierungen (lineares und quadratisches Sondieren, doppeltes Hashing) besitzen also *schlechtere* Performance als die des idealen Hashings durch mögliche Kettenbildung

0.11.3.16 Vergleich des idealen Hashings mit linearem Sondieren

Kosten (Anzahl Proben) für Einfügen und erfolglose Suche

$$C'_\alpha = \frac{1}{1-\alpha} \qquad C'_{\alpha,\text{linSond}} \approx \frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right) \tag{10}$$

Kosten (Anzahl Proben) für Entfernen und erfolgreiche Suche

- Sei $\alpha := \frac{n}{m}$ der „Auslastungsfaktor", d. h. n von m Behältern seien belegt
- Dann gilt:

$$C_\alpha = \frac{1}{\alpha}\ln\frac{1}{1-\alpha} \qquad C_{\alpha,\text{linSond}} \approx \frac{1}{2}\left(1+\frac{1}{1-\alpha}\right) \tag{11}$$

- Beweis: siehe [?]

- Kosten (Anzahl Proben) für Einfügen und *erfolglose* Suche
  - Ideales geschlossenes Hashing: C'_α
  - Lineares Sondieren: $C'_{\alpha,\text{linSond}}$
- Kosten für Entfernen und *erfolgreiche* Suche

© R.-R. Grigat 2003-2019

- Ideales geschlossenes Hashing: C_α
- Lineares Sondieren: $C_{\alpha,\text{linSond}}$

Auslastung	C'_α	$C'_{\alpha,\text{linSond}}$	C_α	$C_{\alpha,\text{linSond}}$
$\alpha = 20\%$	1,3	1,3	1,1	1,1
50%	2,0	2,5	1,4	1,5
80%	5,0	13,0	2,0	3,0
90%	10,0	50,5	2,6	5,5
95%	20,0	200,5	3,2	10,5

Tabelle 18: Lineares Sondieren erfordert mehr Schritte als ideales geschlossenes Hashing; erfolglose Suche ist teurer als erfolgreiche Suche

0.11.3.17 Vergleich des idealen Hashings mit linearem Sondieren

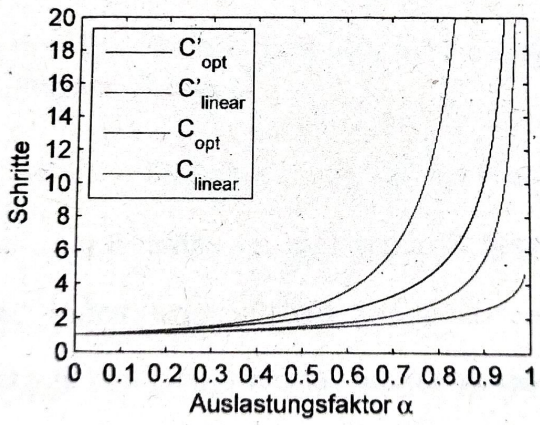


Abbildung 78: Mittlere Zahl der Schritte für erfolgreiche Suche C und erfolglose Suche bzw. Einfügen C' bei idealem (C_{opt}, C'_{opt}) und linearem Hashing (C_{linear}, C'_{linear}). Am schnellsten ist erfolgreiche ideale Suche, am langsamsten Einfügen mit linearer Suche.

0.11.3.18 Kollisionsstrategien

Verallgemeinertes lineares Sondieren:

- Hashfunktionen: $h_i(x) = \big(h(x) + c \cdot i\big) \mod m$
- c und m müssen teilerfremd sein, damit alle Behälter getroffen werden
- c konstant

© R.-R. Grigat 2003-2019

- keine Verbesserung gegenüber $c = 1$ bzgl. Kettenbildung

Quadratisches Sondieren

- Rehashingfunktionen: $h_i(x) = \big(h(x) + i^2\big) \mod m$
- Wahrscheinlichkeit für die Bildung längerer Ketten wird herabgesetzt

0.11.3.19 Doppeltes Hashing

- Idee: zwei voneinander *unabhängige* Hashfunktionen $h(x)$ und $h'(x)$ verwenden
  - Definition der Unabhängigkeit: für zwei Schlüssel x und y seien die Kollisionswahrscheinlichkeiten jeweils $\frac{1}{m}$.

$$P\big(h(x) = h(y)\big) = \frac{1}{m} \quad \text{und} \quad P\big(h'(x) = h'(y)\big) = \frac{1}{m} \tag{12}$$

  h und h' heißen *unabhängig*, wenn für die Wahrscheinlichkeit einer Doppelkollision gilt:

$$P\big(h(x) = h(y) \wedge h'(x) = h'(y)\big) = \frac{1}{m^2} \tag{13}$$

- Zusammengesetzte Hashfunktion: $h_i(x) = \big(h(x) + h'(x) \cdot i^2\big) \mod m,\ i = 0, 1, \ldots$

$\Rightarrow$ Kosten sind von idealem Hashing kaum unterscheidbar

- Problem: Statistisch unabhängige Hashfunktionen müssen gefunden werden

0.11.3.20 Zusammenfassung: Dictionaries mittels Hashing

Hash-Dictionaries

- Zeitkomplexität im besten Fall: $O(1)$
- Zeitkomplexität im schlechtesten Fall: $O(n)$
- wenn Tabellengröße nicht vorab bekannt, dann offenes Hashing verwenden
- wenn Elemente oft gelöscht werden müssen, dann offenes Hashing verwenden

© R.-R. Grigat 2003-2019

- wenn Gesamtzahl der Schlüssel vorab bekannt und wenige Löschoperationen nötig, dann geschlossenes Hashing verwenden
- bei geschlossenem Hashing sollte Auslastung < 80% sein
- Hash-Funktionen sind relativ einfach zu implementieren
- Hash-Funktionen müssen Schlüssel gleichmäßig verteilen
- Nachteil: Verwendete Schlüssel können nicht einfach sortiert gelistet werden

0.11.4 Binäre Suchbäume

0.11.4.1 Implementierung von Dictionaries: Binäre Suchbäume

- Binäre Suchbäume: Weitere effiziente Art, Dictionaries zu implementieren
- Grundidee: Bäume sind geeignet für eine schnelle Suche nach Elementen, d. h. für die Funktionen
  - *insert*
  - *delete*
  - *member*
- Voraussetzung: Bäume müssen Elemente *sortiert* enthalten

0.11.4.2 Eigenschaften eines binären Suchbaumes

Beispiel: Alphabetische Sortierung

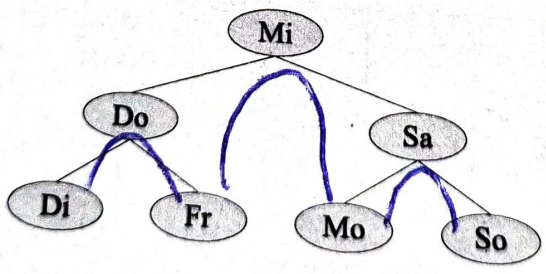


Abbildung 79: Binärer Suchbaum

Binärer Suchbaum

© R.-R. Grigat 2003-2019

- Jeder Knoten hat folgende Eigenschaften:
  - Sei x ein Knoten in einem binären Suchbaum
  - Wenn p_l Knoten im linken Teilbaum von x ist, dann gilt $k_l := \text{key}(p_l) \leq \text{key}(x)$
  - Wenn p_r Knoten im rechten Teilbaum von x ist, dann gilt $s_r := \text{key}(p_r) > \text{key}(x)$
  - Duplikate von x dürfen nur im linken Teilbaum auftreten
- Daten sind in Inorder-Reihenfolge sortiert

0.11.4.3 Suche nach Knoten in binären Suchbäumen

Die Funktion *member*
Suche Knoten p_e mit Schlüssel $k_e = \text{key}(p_e)$ im Baum beginnend bei Wurzel x mit Schlüssel $k_x = \text{key}(x)$

- Falls $\text{key}(p_e) = \text{key}(x)$: Knoten ist gefunden
- Falls $\text{key}(p_e) < \text{key}(x)$:
  - linker Sohn p_l von x vorhanden: Suche p_e beginnend bei Wurzel p_l
  - kein linker Sohn vorhanden: Knoten p_e ist nicht enthalten
- Falls $\text{key}(p_e) > \text{key}(x)$:
  - rechter Sohn p_r von x vorhanden: Suche p_e beginnend bei Wurzel p_r
  - kein rechter Sohn vorhanden: Knoten p_e ist nicht enthalten

0.11.4.4 Einfügen von Knoten in binäre Suchbäume

Die Funktion *insert*
Füge Knoten p_e im Baum beginnend bei Knoten x ein

- Suche p_e im Baum beginnend bei x
- Falls p_e enthalten ist (und Duplikate erlaubt sind): Füge neuen Knoten als linken Sohn von p_e ein

© R.-R. Grigat 2003-2019

- Falls p_e nicht enthalten ist: Erzeuge neues Blatt p_e an der Stelle, an der die Suche erfolglos abgebrochen wurde
- Struktur hängt also von Reihenfolge der eingefügten Elemente ab
- Waren Elemente sortiert, so entartet Suchbaum zu Liste

Insert (Code)

0.11.4.5 Löschen von Elementen aus binären Suchbäumen

Die Funktion *delete*
Lösche Knoten p_e mit Schlüssel $k_e = \text{key}(p_e)$ im Baum

- Definition: *Äquivalenter Wert* $k_{\tilde{e}}$ Kleinster Schlüssel $k_{\tilde{e}} = \text{key}(p_{\tilde{e}})$ im rechten Teilbaum oder größter Schlüssel im linken Teilbaum von p_e
  - Konvention für diese Vorlesung: Wähle als äquivalenten Wert $k_{\tilde{e}}$ kleinsten Schlüssel im rechten Teilbaum
- Suche Knoten p_e anhand k_e im Baum beginnend bei Wurzel x und breche ab, falls p_e nicht enthalten ist
- Wenn p_e gefunden ist, dann unterscheide:
  - p_e ist Blatt: Lösche p_e
  - p_e hat genau einen Sohn: Ersetze p_e durch Sohn
  - p_e hat zwei Söhne: Ersetze Knoten p_e durch Knoten $p_{\tilde{e}}$ mit äquivalentem Wert $s_{\tilde{e}} = \text{key}(p_{\tilde{e}})$ zu p_e und füge ggf. Liste der Knoten mit Schlüssel-Duplikaten von $k_{\tilde{e}}$ links unterhalb ein. Lösche alle kopierten Knoten mit Schlüssel $k_{\tilde{e}}$ an den vorherigen Positionen.

0.11.4.6 Beispiel: Löschen von Elementen aus binären Suchbäumen

Beispiel

© R.-R. Grigat 2003-2019

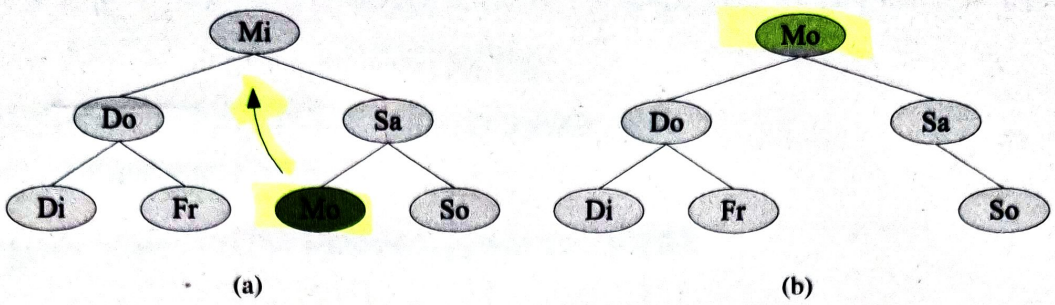


Abbildung 80: Löschen von Elementen aus binären Suchbäumen

- Aufgabe: Lösche „Mi“ aus alphabetisch sortiertem Baum
- Lösung:
  - Identifiziere kleinstes Element im rechten Teilbaum: „Mo“
  - Lösche „Mo“
  - Ersetze „Mi“ durch „Mo“

0.11.4.7 Komplexitätsanalyse bei binären Suchbäumen

Zeitkomplexität der Suche in binären Suchbäumen

- Suchaufwand in Bäumen ist begrenzt durch maximale Pfadlänge
- Bester Fall: Balancierter Baum mit Pfadlänge $O(\log n)$
- Schlechtester Fall: Entarteter Baum mit Pfadlänge $O(n)$
- Durchschnittskomplexität der Suche bei einem Baum, der durch zufälliges Einfügen von Elementen entstanden ist (ohne Beweis): $2 \cdot \ln(2) \cdot \log(n) + \text{const} \Rightarrow O(\log n)$

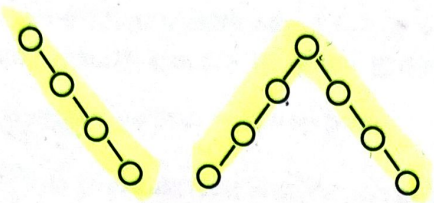
© R.-R. Grigat 2003-2019

0.11.5 AVL-Bäume

0.11.5.1 Balancierte Bäume

Definition: Balancierter Baum
Für jeden Knoten unterscheiden sich die Höhen seiner zwei Teilbäume höchstens um 1

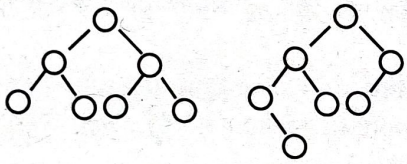
Unbalancierte Bäume



(a) (b)

Abbildung 81: Unbalancierte Bäume

Balancierte Bäume - sind nicht immer vollständig



(a) (b)

Abbildung 82: Balancierte Bäume

0.11.5.2 Implementierung des Dictionary: AVL-Bäume

AVL-Baum

- Der AVL-Baum ist ein balancierter, binärer Suchbaum
- Ein *balancierter* Baum kann nicht zu Liste entarten
- Aufwand für alle drei Dictionary-Operationen garantiert $O(\log n)$
- Benannt nach den zwei Mathematikern Adelson-Velskii und Landis

© R.-R. Grigat 2003-2019

- Wir betrachten nur AVL-Bäume ohne Duplikate der Schlüssel
- Der Fall mit Duplikaten ist komplexer und kann der Fachliteratur entnommen werden

0.11.5.3 Update-Operationen des AVL-Baums

Definition: Update

- Einfügen oder Löschen eines Elements (Knoten p_e)
- Hierdurch kann die Balanciertheit aufgehoben werden

Update: Einfügen eines Elements (*insert*)

- Einfügen des Knotens p_e wie bei binärem Suchbaum
- Neuer Knoten p_e gehört zu allen Teilbäumen, deren Wurzelknoten auf dem Pfad von der AVL-Baumwurzel zu p_e liegen
- Die Höhen aller dieser Teilbäume können um 1 wachsen

Update: Löschen eines Elements (*delete*)

- Löschen des Knotens p_e wie bei binärem Suchbaum
- Betroffen sind alle Teilbäume mit Wurzel auf dem Pfad von der AVL-Baumwurzel zum Vater p_v des gelöschten Knotens p_e
- Höhen dieser Teilbäume können um 1 abnehmen

0.11.5.4 Rebalancieren

Rebalancieren

- notwendig falls durch eine Update-Operation die Balance verletzt wird
- Zwei Fälle:
  a) durch eine Einfügung aus der Balance geraten
  b) durch eine Löschoperation aus der Balance geraten

© R.-R. Grigat 2003-2019

Erforderliche Rebalancieroperationen

- Einzelne Rotation
- Doppelrotation

0.11.5.5 Einzelrotation (Pivot P) oder Doppelrotation (P1, P2)?

Pivot, Rotator, innerer/äußerer Teilbaum

- *Pivotknoten P oder P2* ist der tiefste Knoten ohne Balance
- Der *Rotator R* ist dasjenige Kind von Pivot P, P1 oder P2, welches die Wurzel des größeren Teilbaums von P ist
- Ist R *rechter* Nachfolger von P oder P2, so nennen wir den rechten Teilbaum unter R den *äußeren* Teilbaum C und den linken Teilbaum den *inneren* Teilbaum B. Die Wurzeln von B und C sind Kinder von R und Geschwister.
- Ist Rotator R dagegen *linker* Nachfolger von Pivot P oder P2, so ist es umgekehrt (innerer Teilbaum B rechts unterhalb R, äußerer Teilbaum C links unterhalb R).
- Hat der äußere Teilbaum C größere Höhe als der innere Teilbaum B, dann erfolgt *Einzelrotation* mit Pivot P
- Hat dagegen der innere Teilbaum B die größere Höhe, dann erfolgt *Doppelrotation* mit Pivots P1 und P2

0.11.5.6 Rotationsoperation

Einzelrotation

- Pivot P sinkt und Rotator R steigt um je eine Höhenstufe
- Alle Teilbäume folgen Pivot bzw. Rotator
- Ausnahme: Innerer Teilbaum behält Höhenstufe und wechselt vom Rotator zum Pivot.
- War innerer Teilbaum *linker*(rechter) Nachfolger des Rotators, so wird er *rechter*(linker) Nachfolger des Pivots.

© R.-R. Grigat 2003-2019

Doppelrotation

- besteht aus zwei gegenläufigen Einzelrotationen
- Die erste dieser Einzelrotationen stellt die Voraussetzung der eigentlichen (zweiten) Einzelrotation her, indem der äußere Teilbaum die größere Höhe erhält.
- Rotatorknoten R1, R2 beider Einzelrotationen identisch.
- Pivotknoten *P1* und *P2* beider Einzelrotationen *unterschiedlich*, beide sind nachher Söhne des Rotators.

0.11.5.7 Rebalancieren: Durch Einfügen aus der Balance geraten

Äußerer Teilbaum C nach Einfügen höher als innerer Teilbaum B

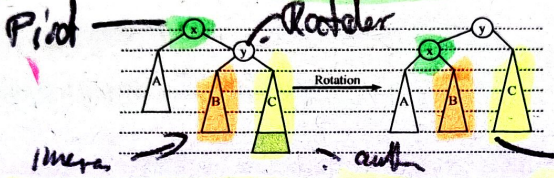


Abbildung 83: Einzelrotation, Pivot x sinkt, Rotator y steigt, innerer Teilbaum wechselt von Rotator zu Pivot

- Wiederherstellung der Balance durch *Einzelrotation*

Innerer Teilbaum B nach Einfügen höher als äußerer Teilbaum C

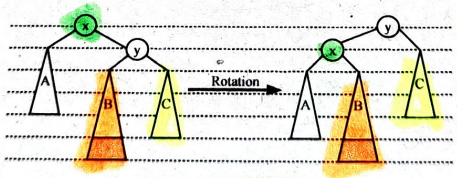


Abbildung 84: Gegenbeispiel: Einzelrotation ist *keine* Lösung (Pivot x, Rotator y)

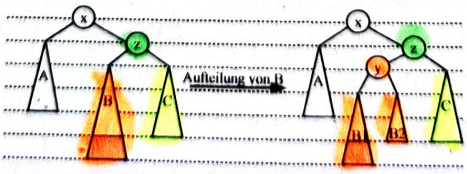


Abbildung 85: Aufteilen des größeren Teilbaums B in zwei Teilbäume (B1, B2)

© R.-R. Grigat 2003-2019

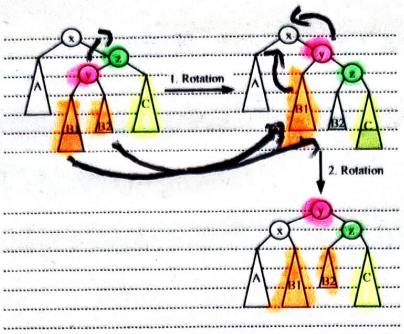
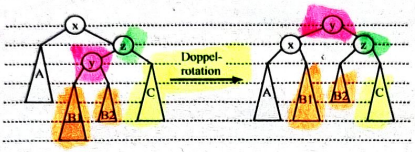
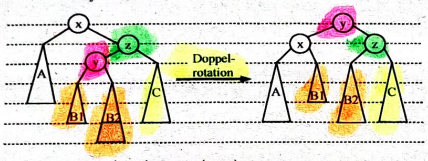


Abbildung 86: Doppelrotation. Erste Rotation erfolgt mit Pivot P1=z und Rotator R1=y, *obwohl* Teilbaum mit Wurzel z balanciert ist.



(a) $h(B1) > h(B2)$: Fall 1 von 2



(b) $h(B1) < h(B2)$: Fall 2 von 2

Abbildung 87: Doppelrotation: 2 Fälle

- Einzelne Rotation kann die Balance *nicht* wiederherstellen
- Wiederherstellung der Balance durch *Doppelrotation*

Ergebnis für Update durch Einfügen:

- Es genügt eine Rotation oder Doppelrotation, um Balance nach *Einfügen* eines Elements wiederherzustellen

Rotation

- *inorder* Reihenfolge bleibt erhalten: $< A, x, B, y, C >$

© R.-R. Grigat 2003-2019

- Ordnungseigenschaft des Suchbaumes bleibt erhalten

Doppelrotation

1. Pivot P1=z sinkt, Rotator y steigt
2. Pivot P2=x sinkt, Rotator y steigt

Lösungsweg identisch, falls $h(B1) < h(B2)$

0.11.5.8 Rebalancieren: Durch Löschen aus der Balance geraten

***Äußerer* Teilbaum C nach Löschen höher als innerer Teilbaum B**

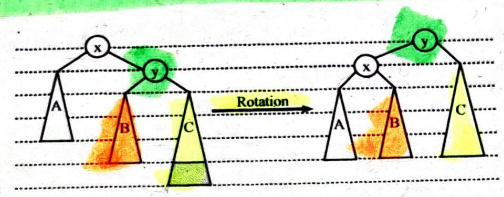


Abbildung 88: Einzelrotation gegen Uhrzeigersinn

- Wiederherstellung der Balance durch *Einzelrotation*
- Pivot P=x sinkt, Rotator y steigt
- Höhe um 1 vermindert

Äußerer* Teilbaum C und innerer Teilbaum B nach Löschen *gleich hoch

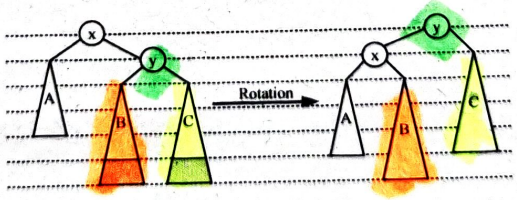
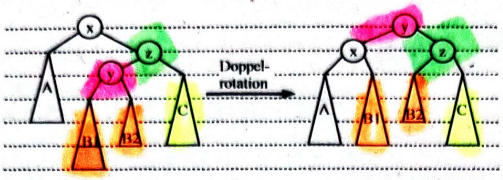


Abbildung 89: Einzelrotation, Pivot x, Rotator y

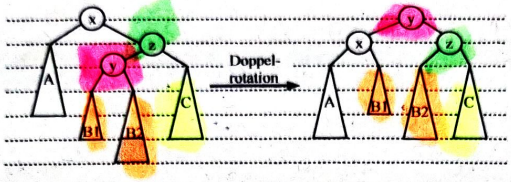
- Wiederherstellung der Balance durch *Einzelrotation*
- Pivot x, Rotator y
- Höhe bleibt gleich

© R.-R. Grigat 2003-2019

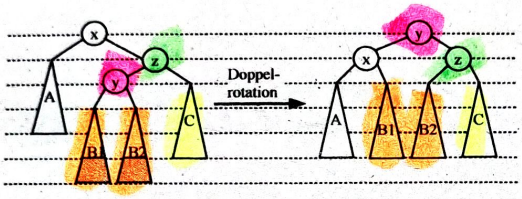
Innerer Teilbaum B nach Löschen höher als äußerer Teilbaum C



(a) $h(B1) > h(B2)$: Fall 1 von 3



(b) $h(B1) < h(B2)$: Fall 2 von 3



(c) $h(B1) = h(B2)$: Fall 3 von 3

Abbildung 90: Doppelrotation: 3 Fälle

- Bereits gezeigt bei Rebalancierung nach Einfügen: Eine einzelne Rotation kann die Balance dieser Konstellation *nicht* wiederherstellen
- Wiederherstellung der Balance durch *Doppelrotation*
- Höhe wird um 1 vermindert

Ergebnis für Update durch Löschen:

- Es genügt eine Rotation oder Doppelrotation, um Balance eines Teilbaumes nach Löschen eines Elements wiederherzustellen
- Falls sich dadurch Höhe ändert, müssen *auch Vorgängerteilbäume* ggf. rebalanciert werden
- Balancewert b ist die Differenz aus Höhe des linken Teilbaumes minus Höhe des rechten Teilbaumes

© R.-R. Grigat 2003-2019

- $b \in \{-1,0,1\} \Rightarrow$ Knoten ist balanciert
- $b \in \{-2,2\} \Rightarrow$ Knoten ist unbalanciert

- Für Implementierung eines AVL-Baumes sollten Höhe oder Balancewert in jedem Knoten gespeichert werden

0.11.5.9 Beispiel zu AVL-Bäumen

Beispiel:

- Einfügen der Wochentagsnamen in einen AVL-Baum, alphabetische Sortierung
- Speicherung des Balancewertes b in jedem Knoten

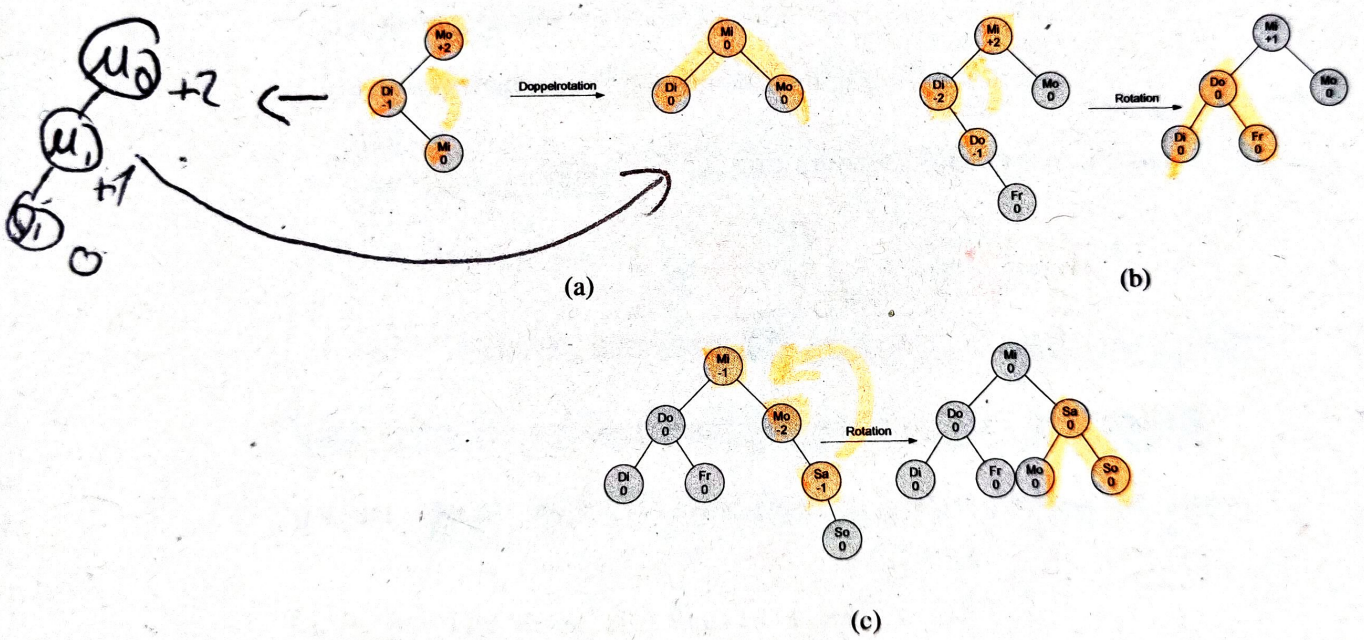


Abbildung 91: Beispiel zu AVL-Bäumen

© R.-R. Grigat 2003-2019

0.11.6 Optimale binäre Suchbäume

0.11.6.1 Einleitung

Binärer Suchbaum

- Binärer Suchbaum ist geeignete Datenstruktur für Datentyp Dictionary, d. h. für die Funktionen
  - *insert*
  - *delete*
  - *member*
- Weiterentwicklung des binären Suchbaums, um Entartung zu linearer Liste zu vermeiden: AVL-Bäume
- Sind individuelle Wahrscheinlichkeiten gegeben, mit denen die Elemente gesucht werden, kann optimale Anordnung der Elemente bzgl. Suchaufwand bestimmt werden $\Rightarrow$ *Optimaler binärer Suchbaum*

0.11.6.2 Eigenschaften eines optimalen binären Suchbaumes

Gegeben:

- Sequenz $K = \langle k_1, k_2, \ldots, k_n \rangle$ von n paarweise verschiedenen Schlüsseln mit $k_1 < k_2 < \ldots < k_n$
- Berücksichtigen, dass auch nach Schlüsseln gesucht wird, die nicht im Baum vorkommen
  - Dazu $n+1$ Dummy-Schlüssel $k_{d_0}, k_{d_1}, k_{d_2}, \ldots, k_{d_n}$ einfügen
  - k_{d_0} steht für alle Schlüssel kleiner als k_1
  - k_{d_n} steht für alle Schlüssel größer als k_n
  - k_{d_i} steht für alle Schlüssel zwischen k_i und k_{i+1} $\forall\, i = 1, 2, \ldots, n-1$
- Wahrscheinlichkeiten P_i für Suche nach Schlüssel k_i
- Wahrscheinlichkeiten P_{d_i} für Suche nach Dummy-Schlüssel k_{d_i}

Es gilt:

© R.-R. Grigat 2003-2019

- Jeder Schlüssel k_i ist in innerem Knoten p_i gespeichert
- jeder Dummy-Schlüssel k_{d_i} ist in Blattknoten p_{d_i} gespeichert
- Suche ist entweder erfolgreich (Schlüssel k_i wird gefunden) oder erfolglos (Dummy-Schlüssel k_{d_i} wird gefunden)

$$\Longrightarrow \quad \sum_{i=1}^{n} P_i + \sum_{i=0}^{n} P_{d_i} = 1 \tag{14}$$

- Erwartete Kosten $E\{T\}$ für Suche in optimalem binärem Suchbaum T:
  - Sei $\text{tiefe}_T(p)$ die Tiefe eines Knotens p in Baum T
  - Kosten für Suche eines Knotens p_i ergeben sich aus Anzahl der hierzu überprüften Knoten, also $\text{tiefe}_T(p_i)+1$
  - Für den Erwartungswert der Kosten $E\{T\}$ für Suche beliebiger Knoten folgt

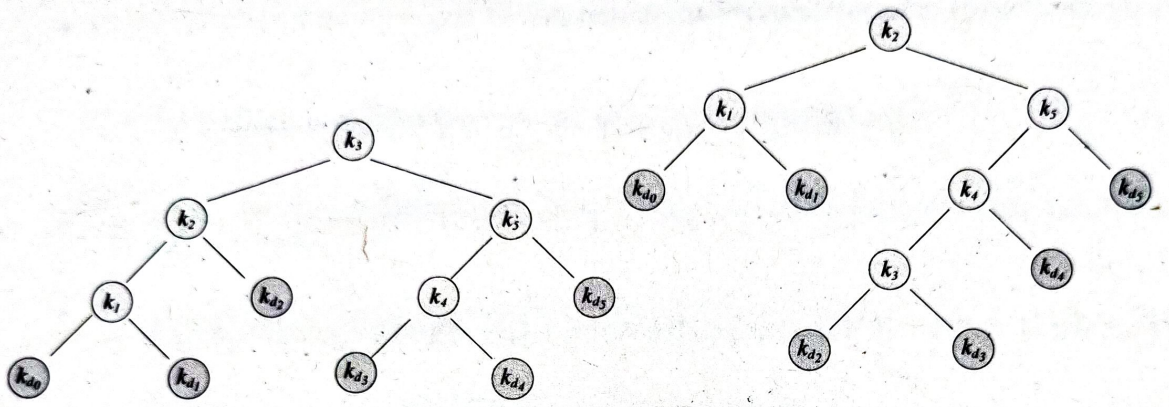
$$\begin{aligned} E\{T\} &= \sum_{i=1}^{n} (\text{tiefe}_T(p_i)+1)\cdot P_i + \sum_{i=0}^{n} (\text{tiefe}_T(p_{d_i})+1)\cdot P_{d_i} \\ &= 1 + \sum_{i=1}^{n} \text{tiefe}_T(p_i)\cdot P_i + \sum_{i=0}^{n} \text{tiefe}_T(p_{d_i})\cdot P_{d_i} \end{aligned} \tag{15}$$

© R.-R. Grigat 2003-2019

Beispiel

i	0	1	2	3	4	5
P_i		0.15	0.10	0.05	0.10	0.20
P_{d_i}	0.05	0.10	0.05	0.05	0.05	0.10

Tabelle 19: Beispiel: Wahrscheinlichkeiten P_i und P_{d_i} für innere Knoten p_i und Blätter p_{d_i}



(a) höhenbalanciert, aber *suboptimal*, erwartete Kosten für Suche $E\{T\} = 2.85$

(b) *optimal*, erwartete Kosten für Suche $E\{T\} = 2.75$, Herleitung folgt

Abbildung 92: Binäre Suchbäume

0.11.6.3 Schritt 1/4: Struktur eines optimalen binären Suchbaums

Eigenschaften von Teilbäumen eines *allgemeinen* binären Suchbaums

- Jeder Teilbaum muss Schlüssel aus *zusammenhängendem* Bereich $k_i, \ldots, k_j$ mit $1 \leq i \leq j \leq n$ und $k_i < k_{i+1} < \ldots < k_j$ enthalten
- Teilbaum, der die Schlüssel $k_i, \ldots, k_j$ enthält, muss Dummy-Schlüssel $k_{d_{i-1}}, \ldots, k_{d_j}$ als Blätter enthalten

Optimale Teilstruktur

- In *optimalen* binären Suchbäumen sind *alle Teilbäume ebenfalls* optimal.
- Anders formuliert: Sei T' mit den Schlüsseln $k_i, \ldots, k_j$ ein Teilbaum des optimalen binären Suchbaums T

© R.-R. Grigat 2003-2019

⇒ Teilbaum T' ist ebenfalls *optimal* für Teilproblem mit den Schlüsseln $k_i, \ldots, k_j$

- Beweis: Sei T'' Teilbaum mit gleicher Schlüsselmenge $k_i, \ldots, k_j$ und $E\{T''\} < E\{T'\}$
- Dann könnte T' durch T'' ersetzt werden
- Die Gesamtkosten $E\{T\}$ würden sinken ⇒ Widerspruch

- **Suche nach Optimum:**
  - Seien Schlüssel $k_i, \ldots, k_j$ gegeben
  - Sei Schlüssel k_r mit $(i \leq r \leq j)$ in Wurzel p_r des optimalen Suchbaums gespeichert
  - Linker Teilbaum der Wurzel p_r enthält Schlüssel $k_i, \ldots, k_{r-1}$ (und Dummy-Schlüssel $k_{d_{i-1}}, \ldots, k_{d_{r-1}}$)
  - Rechter Teilbaum enthält Schlüssel $k_{r+1}, \ldots, k_j$ (und Dummy-Schlüssel $k_{d_r}, \ldots, k_{d_j}$)

  ⇒ Alle Kandidaten p_r für Wurzelknoten mit $(i \leq r \leq j)$ und deren zugehörige optimale Such-Teilbäume mit Knoten $k_i, \ldots, k_{r-1}$ bzw. $k_{r+1}, \ldots, k_j$ vergleichen und optimalen Kandidaten wählen

Anmerkung zu „leeren" Teilbäumen:

- Sei p_i Wurzelknoten, also $r = i$
- Nach obiger Beschreibung enthält linker Teilbaum von p_i die Knoten $p_i, \ldots, p_{i-1}$
- Konvention: $p_i, \ldots, p_{i-1}$ enthält nur den Dummy-Knoten $p_{d_{i-1}}$
- Analog: Sei p_j Wurzelknoten, also $r = j$
- Rechter Teilbaum von p_j enthält die Knoten $p_{j+1}, \ldots, p_j$
- Konvention: $p_{j+1}, \ldots, p_j$ enthält nur den Dummy-Knoten p_{d_j}

0.11.6.4 Schritt 2/4: Rekursive Formulierung des Problems

Teilprobleme:

- Berechne optimalen binären Suchbaum, der die Schlüssel $k_i, \ldots, k_j$ enthält, $\forall\, 1 \leq i < j \leq n$

© R.-R. Grigat 2003-2019

- Sei $e[i,j]$ der Erwartungswert der Kosten für Suche in optimalem binärem Suchbaum, der die Schlüssel $k_i,\ldots,k_j$ enthält
- Ziel: $E\{T\} = e[1,n]$ berechnen

Auswahl eines Knoten p_r als Wurzel für optimale Teilbäume

- Durch Einfügen eines Teilbaums an neuen Wurzelknoten p_r erhöhen sich die Tiefen aller Knoten im Teilbaum um 1
- Aus Gleichung 15 folgt: Der Erwartungswert der Kosten für den Teilbaum mit den Schlüsseln $k_i,\ldots,k_j$ erhöht sich um

$$\sum_{l=i}^{j} P_l + \sum_{l=i-1}^{j} P_{d_l} =: w[i,j] \tag{16}$$

- Es gilt also:

$$e[i,j] = P_r + (e[i,r-1] + w[i,r-1]) + (e[r+1,j] + w[r+1,j]) \tag{17}$$

- Mit $w[i,j] = w[i,r-1] + P_r + w[r+1,j]$ gilt:

$$e[i,j] = e[i,r-1] + e[r+1,j] + w[i,j] \tag{18}$$

- In jedem Schritt den Knoten p_r mit den niedrigsten erwarteten Kosten auswählen

Rekursive Beschreibung des optimalen binären Suchbaums

Initialisierung:
$$e[i,j] = P_{d_{i-1}} \quad \text{falls } j = i-1 \quad \forall\, i = 1,\ldots,n \tag{19}$$

Induktion:
$$e[i,j] = \min_{i \le r \le j} \{e[i,r-1] + e[r+1,j] + w[i,j]\}$$
$$\text{falls } i \le j \tag{20}$$

Terminierung:
$$e[1,n] = \min_{1 \le r \le n} \{e[1,r-1] + e[r+1,n] + w[1,n]\} \tag{21}$$

© R.-R. Grigat 2003-2019

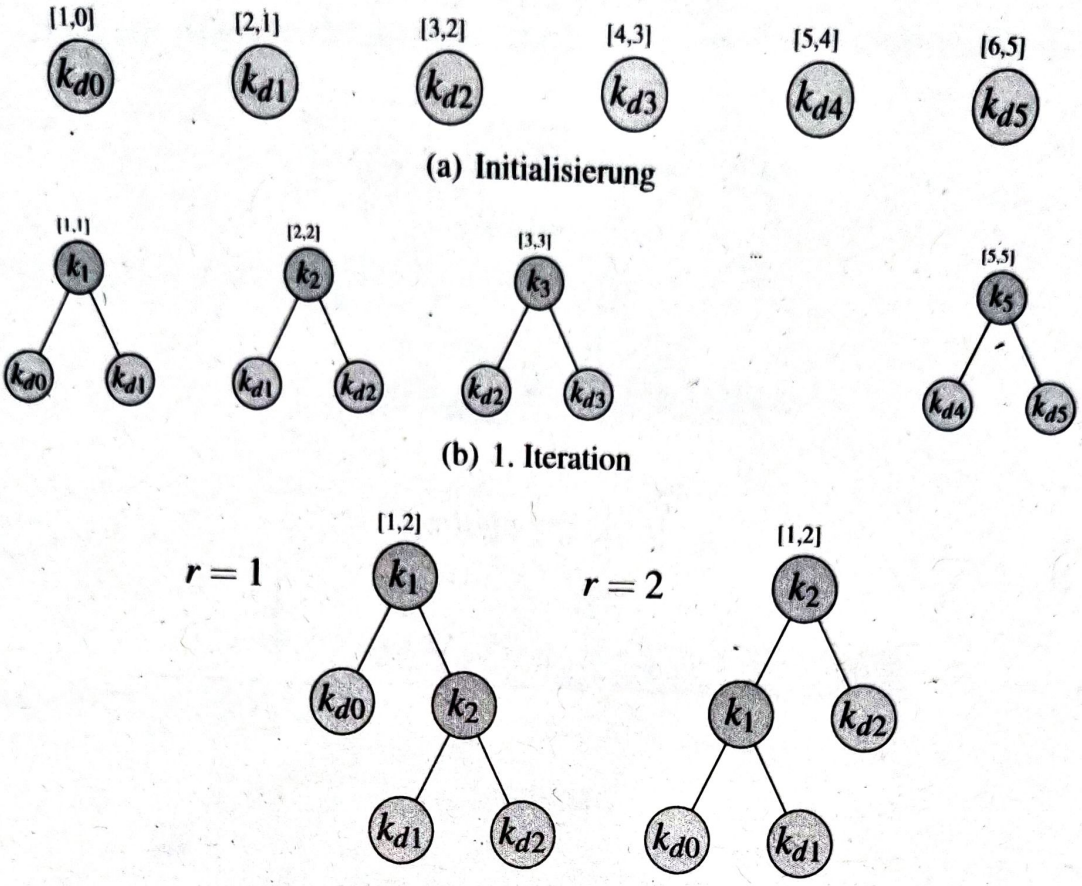


Abbildung 93: Welche Wahl des Wurzelknotens ist optimal? Rekursiver Aufbau des optimalen, binären Suchbaums (hier für die Schlüssel k_1 und k_2).

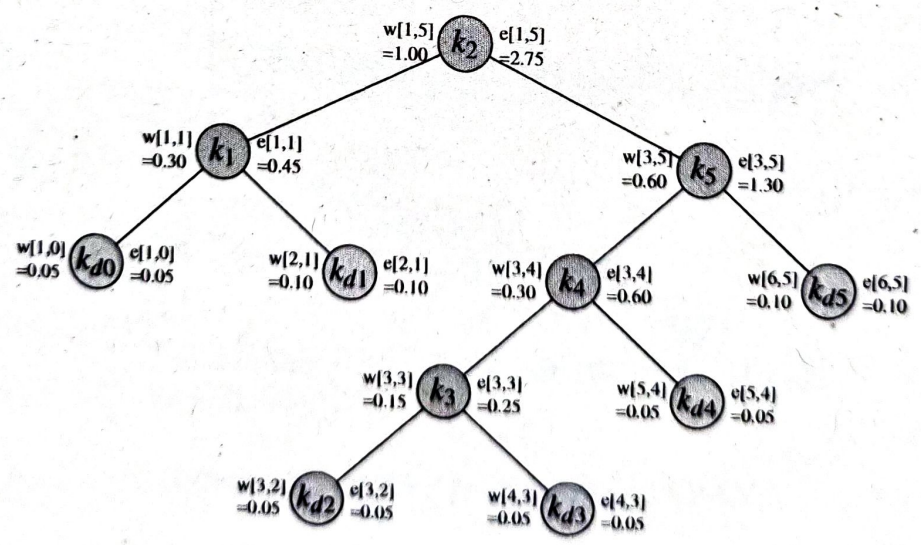


Abbildung 94: Beispiel optimaler Lösung für binären Suchbaum

© R.-R. Grigat 2003-2019

0.11.6.5 Schritt 3/4: Berechnung des optimalen Wertes (Suchkosten)

Bottom-up: Iterative Berechnung minimaler Suchkosten mit Hilfe von Tabellen

- Speichere Werte $e[i,j]$ in Tabelle $e[1\ldots n+1,0\ldots n]$
  - Erster Index muss bis $n+1$ laufen, da $e[n+1,n]$ die Kosten für leeren Teilbaum mit Dummy-Schlüssel k_{d_n} sind
  - Zweiter Index muss bei 0 starten, da $e[1,0]$ die Kosten für leeren Teilbaum mit Dummy-Schlüssel k_{d_0} sind
  - Es werden nur Tabelleneinträge für $j \geq i-1$ benötigt
- Tabelle wurzel$[i,j]$ anlegen, die Wurzel für optimalen Teilbaum mit Schlüsseln $k_i,\ldots,k_j$ speichert
  - Es werden nur Einträge für $1 \leq i \leq j \leq n$ benötigt
- Tabelle $w[1\ldots n+1,0\ldots n]$ für Werte $w[i,j]$ anlegen
  - Start: Berechne $w[i,i-1] = P_{d_{i-1}} \quad \forall\, 1 \leq i \leq n+1$
  - Für jede Folgenlänge $s = j-i+1 \in \{1\ldots n\}$ und für $i \in \{1\ldots n+1-s\}$ berechne $w[i,j] = w[i,j-1] + P_j + P_{d_j}$ mit $j = i-1+s$, vgl. Gleichung 16

Komplexität der Berechnung des optimalen Suchbaums: $O(n^3)$

0.11.6.6 Schritt 4/4: Konstruktion der Lösung (optimaler Suchbaum)

Top-Down: Konstruktion des optimalen binären Suchbaumes

- Aus Tabelle an Position wurzel$[1,N]$ Indexwert r ablesen, p_r als Wurzelknoten verwenden
- Rekursiv aus Tabelle an den Positionen wurzel$[1,r-1]$, wurzel$[r+1,N]$ nächste Wurzelknoten bestimmen

© R.-R. Grigat 2003-2019

0.11.6.7 Beispiel: Berechnung minimaler Suchkosten mit Tabellen

i	0	1	2	3	4	5
P_i		0.15	0.10	0.05	0.10	0.20
P_{d_i}	0.05	0.10	0.05	0.05	0.05	0.10

Tabelle 20: Wahrscheinlichkeiten P_i, P_{d_j}

j \ i	1	2	3	4	5	6
5	1.0	0.80	0.60	0.50	0.35	0.10
4	0.70	0.50	0.30	0.20	0.05	
3	0.55	0.35	0.15	0.05		
2	0.45	0.25	0.05			
1	0.30	0.10				
0	0.05					

Tabelle 21: $w[i, j]$

j \ i	1	2	3	4	5	6
5	2.75	2.00	1.30	0.90	0.50	0.10
4	1.75	1.20	0.60	0.30	0.05	
3	1.25	0.70	0.25	0.05		
2	0.90	0.40	0.05			
1	0.45	0.10				
0	0.05					

Tabelle 22: $e[i, j]$

j \ i	1	2	3	4	5
5	2	4	5	5	5
4	2	2	4	4	
3	2	2	3		
2	1	2			
1	1				

Tabelle 23: wurzel$[i, j]$

© R.-R. Grigat 2003-2019

- $w[i,i-1] = P_{d_{i-1}} \; \forall \, 1 \leq i \leq n+1$
- $s = 1 \ldots n, \; i = 1 \ldots n+1-s \;\; w[i,j] = w[i,s-1+i] = w[i,j-1] + P_j + P_{d_j}$

- $e[i,j] = P_{d_{i-1}} \forall \; j = i-1$
- $e[i,j] = \min\limits_{i \leq r \leq j} \{e[i,r-1] + e[r+1,j] + w[i,j]\} \; \forall \, i \leq j$

- $e[1,1] = \min(0.45)$
- $e[2,2] = \min(0.40)$
- $e[3,3] = \min(0.25)$
- $e[4,4] = \min(0.30)$
- $e[5,5] = \min(0.50)$
- $e[1,2] = \min(0.90, 0.95)$
- $e[2,3] = \min(0.70, 0.80)$
- $e[3,4] = \min(0.65, 0.60)$
- $e[4,5] = \min(1.05, 0.90)$
- $e[1,3] = \min(1.30, 1.25, 1.50)$
- $e[2,4] = \min(1.20, 1.20, 1.25)$
- $e[3,5] = \min(1.55, 1.35, 1.30)$
- $e[1,4] = \min(1.95, 1.75, 1.90, 2.00)$
- $e[2,5] = \min(2.20, 2.10, 2.00, 2.10)$
- $e[1,5] = \min(3.05, 2.75, 2.80, 2.75, 2.85)$

0,05 + 0,40 + 0,45 ; 0,45 + 0,05 = m(/ +0,45)

© R.-R. Grigat 2003-2019

0.11.6.8 Beispiel: Konstruktion eines optimalen binären Suchbaums

- wurzel$[1,5] = 2$ (Abb. 95(a))

- wurzel$[1,1] = 1$ (Abb. 95(b))

- wurzel$[3,5] = 5$ (Abb. 95(d))

- wurzel$[3,4] = 4$ (Abb. 95(e))

- wurzel$[3,3] = 3$ (Abb. 95(f))

© R.-R. Grigat 2003-2019

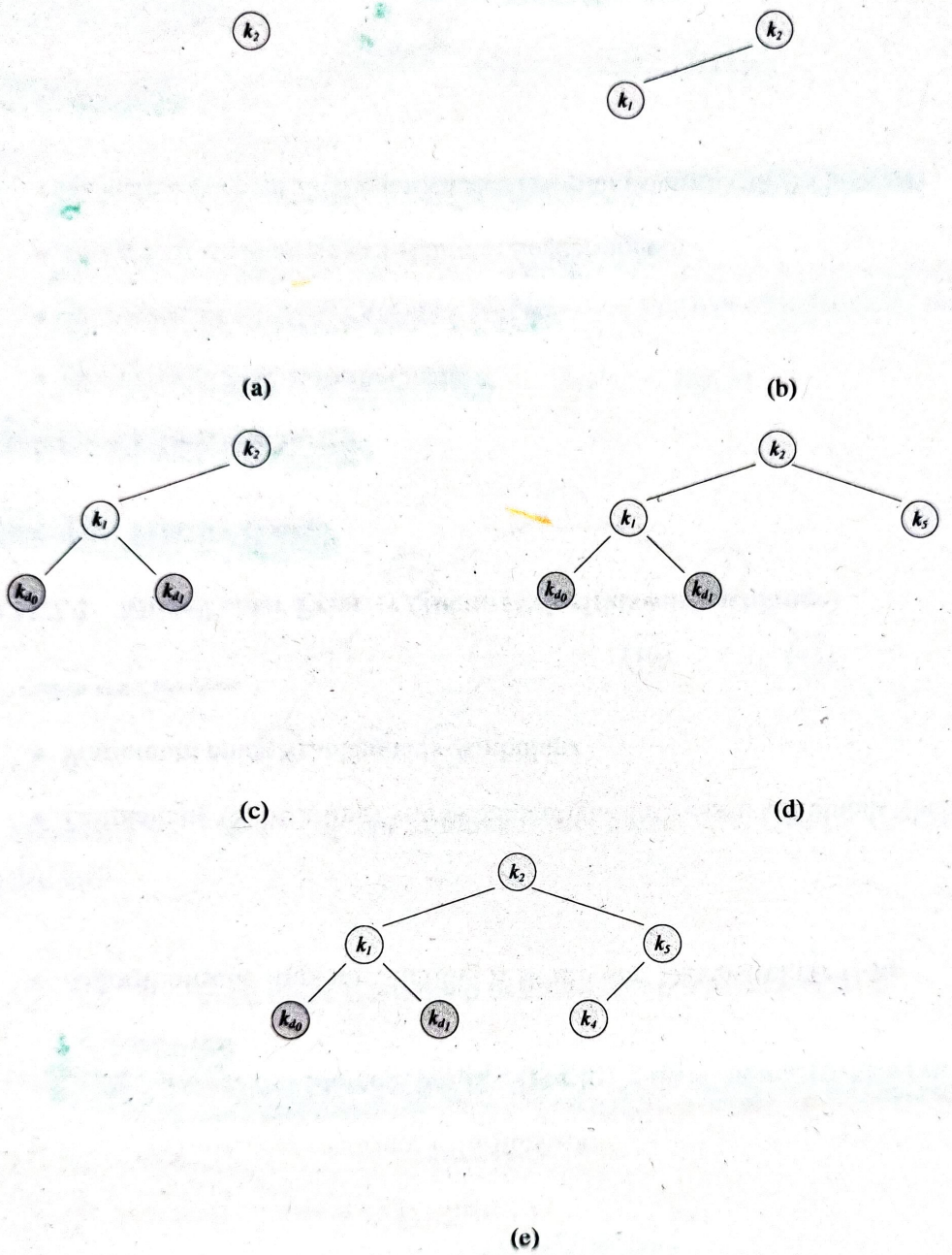


Abbildung 95: Konstruktion eines optimalen Suchbaums

© R.-R. Grigat 2003-2019

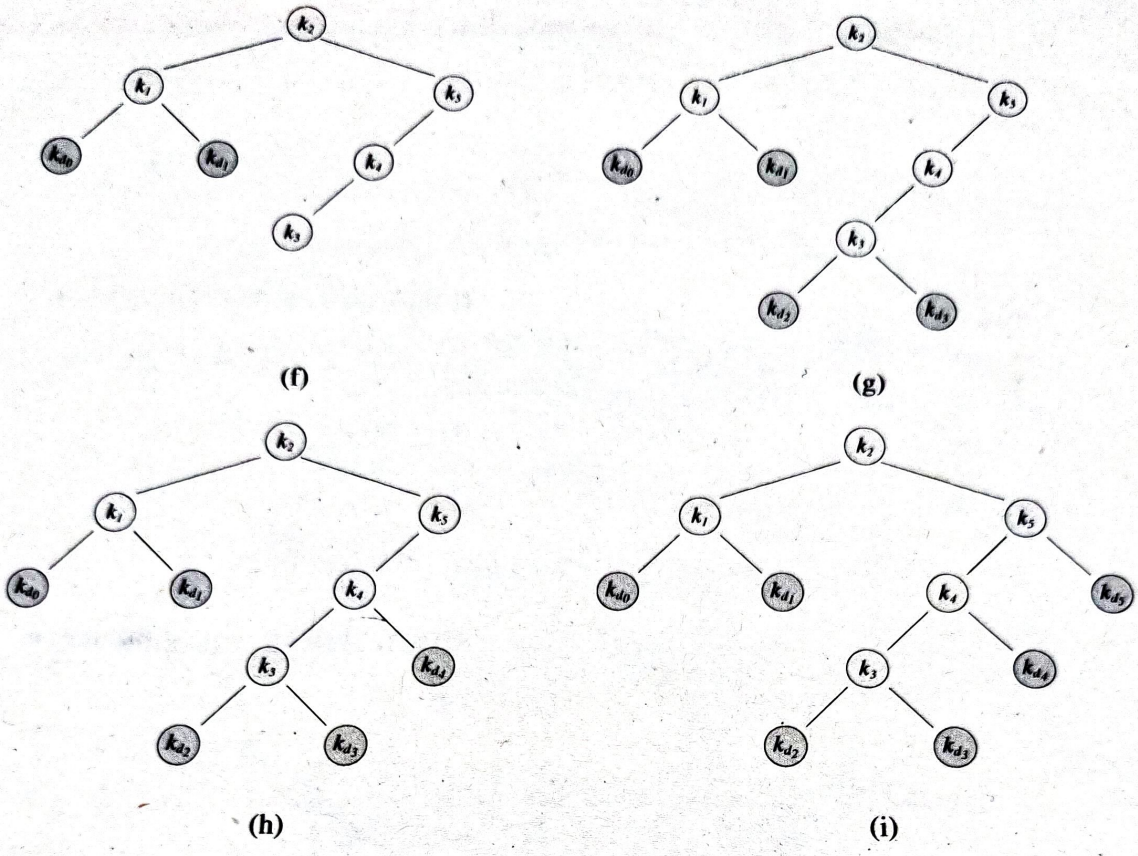


Abbildung 95: Konstruktion eines optimalen Suchbaums

© R.-R. Grigat 2003-2019

0.11.7 Priority Queues

0.11.7.1 Datentyp Priority Queue (Prioritätswarteschlange)

Eigenschaften

- Queue mit Sortierung der Elemente nach Schlüssel
- die Schlüssel enthalten *Prioritätswerte*
- hohe Priorität $\Longleftrightarrow$ niedriger Prioritätswert
- es wird jeweils das Element mit höchster Priorität entnommen, also mit kleinstem Schlüsselwert
- Algorithmische Implementierung z. B. mit der Datenstruktur *Heap*

Beispiele

- Zeitplanung (Scheduling) von nebenläufigen Prozessen auf einem Rechner
- Warteraum einer Krankenhaus-Ambulanz

Example: KP33_2_priorityQueue

0.11.7.2 Modell einer Priority Queue (Prioritätswarteschlange)

Datentyp: Priority Queue

Funktionen (Beschreibung):

- *new* erzeugt leere Priority Queue
- *isempty* prüft, ob Priority Queue leer ist
- *insert* fügt Element unter Erhalt der Sortierung ein
- *deletemin* entfernt Element höchster Priorität (kleinster Prioritätswert)

Datentypen:

$$\begin{array}{l} bool \\ elemSet \\ pqueue \quad = \quad M(elemSet) \end{array}$$

© R.-R. Grigat 2003-2019

- *M*: *Multimenge* (Menge, bei der Elemente aus der Menge elemSet mehrfach auftreten können; Multiset, Bag)
- Mengenoperationen besitzen modifizierte Bedeutung gegenüber Menge
- Notation Multimenge aus Elementen: $\{|\, p_1, \ldots, p_n \,|\}$
- Multimenge aus Prioritätswerten: $\{|\, 1,1,2,3,4,4,5 \,|\}$

Funktionen (Definition):

$$
\begin{array}{lll}
\textit{new:} & \rightarrow & \textit{pqueue} \\
\textit{new} & = & \varnothing \\
 & & \\
\textit{isempty:}\quad \textit{pqueue} & \rightarrow & \textit{bool} \\
\textit{isempty}(q_p) & = & (q_p = \varnothing) \\
 & & \\
\textit{insert: pqueue} \times \textit{elemSet} & \rightarrow & \textit{pqueue} \\
\textit{insert}(q_p, e) & = & q_p \cup \{|\, e \,|\} \\
 & & \\
\textit{min:}\quad \textit{pqueue} & \rightarrow & \textit{elemSet} \\
\textit{min}(q_p) & = & e_{\min} \text{ mit } \mathrm{key}(e_{\min}) \leq \mathrm{key}(e)\ \forall e \in q_p \\
 & & \\
\textit{deletemin:}\quad \textit{pqueue} & \rightarrow & \textit{pqueue} \times \textit{elemSet} \\
 & & \begin{cases} (q'_p, e) & \text{mit } e = \textit{min}(q_p) \\ & \text{und } q'_p = q_p \setminus \{|\, e \,|\} \\ & \text{falls } q_p \neq \varnothing \\ \text{undef.} & \text{sonst} \end{cases} \\
\textit{deletemin}(q_p) & = & \\
\end{array}
$$

© R.-R. Grigat 2003-2019

0.11.8 Heaps

0.11.8.1 Implementierung von Priority Queues mit Heaps

Definition: Heap (Haufen)

- Links-vollständiger, binärer Baum
- Partiell geordneter Baum: Jeder Pfad von der Wurzel zu einem Blatt ist nach Schlüssel $k_i = \text{key}(p_i)$ der Knoten p_i sortiert. Für jeden Teilbaum T' mit Wurzelknoten x gilt:

$$\forall p \in T' : \quad key(x) \leq key(p)$$

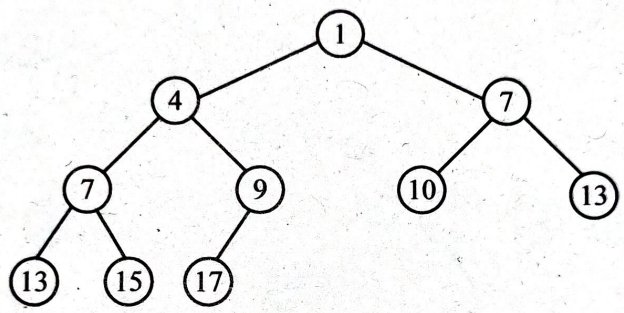


Abbildung 96: Darstellung der Schlüssel-Multimenge $\{|1,4,7,7,9,10,13,13,15,17|\}$ als Heap

0.11.8.2 Einfügen eines Knotens p in einen Heap h mittels SiftUp

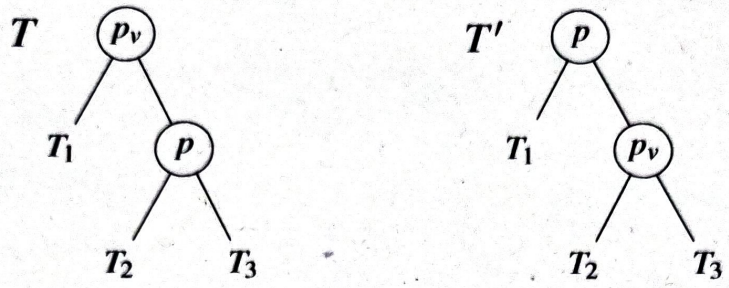
insert(h,p)

```
1 Erzeugen eines neuen Knotens p mit Schlüssel k = key(p)
2 Einfügen von p auf erster freier Position in unterster Ebene (von links)
3 Sei p_v jeweils der Vater von p
4 while ( (p_v existiert) and (key(p) < key(p_v)) )
5 {
6         Vertauschen der Knoten p_v und p (=: SiftUp)
7 }
```

© R.-R. Grigat 2003-2019

0.11.8.3 Korrektheit des Einfüge-Algorithmus

Beweis der Korrektheit von *insert*



- p ist der aufsteigende Knoten, p_v ist Vater
- Falls $\text{key}(p) < \text{key}(p_v)$:
  - Heap-Eigenschaft noch verletzt, Vertauschen notwendig
  - $\text{key}(p_v)$ ist minimal bzgl. T_1, T_2 und T_3.
  - wegen $\text{key}(p) < \text{key}(p_v)$ ist $\text{key}(p)$ minimal bzgl. T_1, T_2 und T_3
  - damit ist für T' Heap-Eigenschaft erfüllt

© R.-R. Grigat 2003-2019

0.11.8.4 Beispiel zu Heap

Beispiel: Einfügen des Elements 3 in Heap

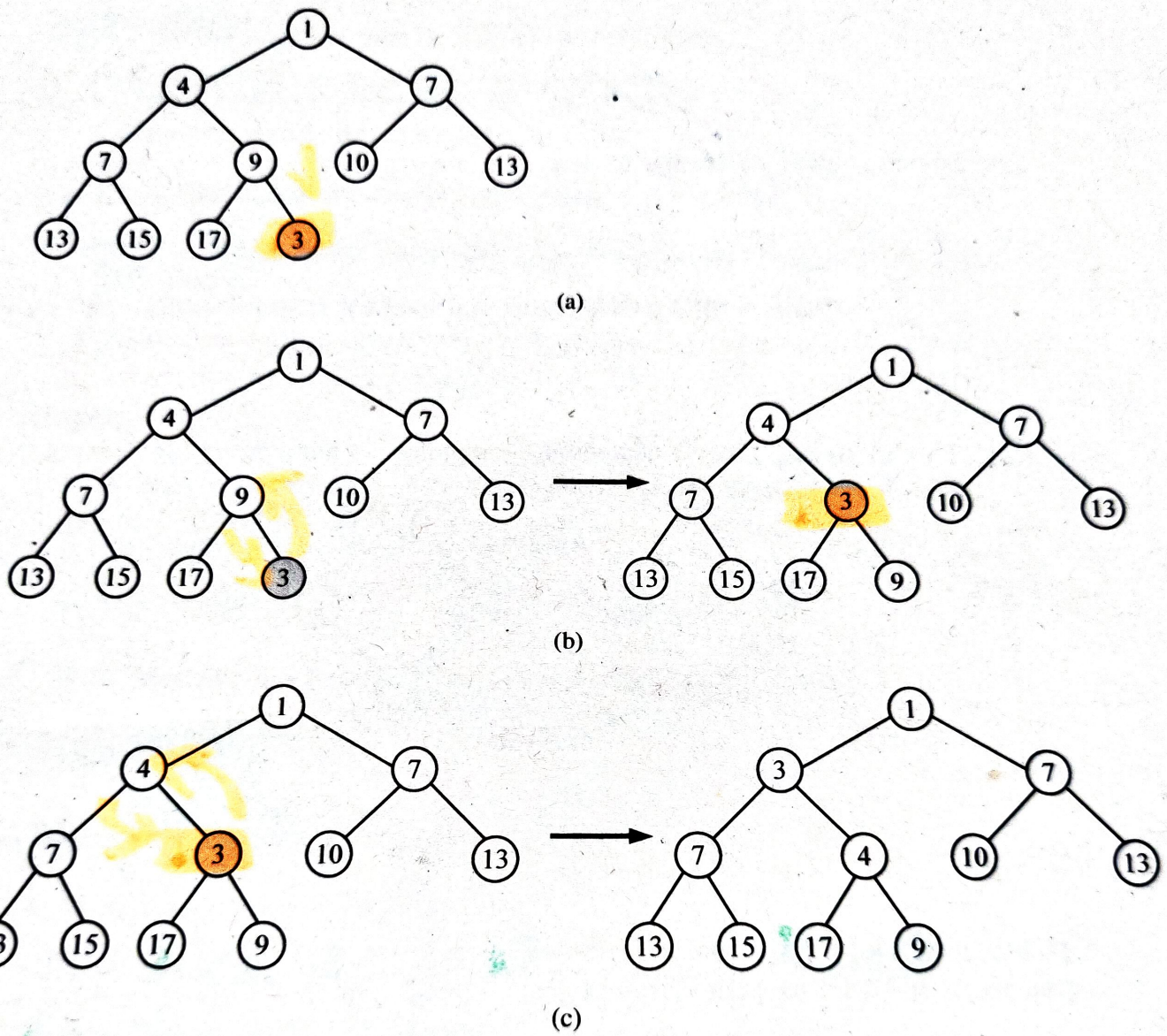


Abbildung 97: Beispiel zu Heap

© R.-R. Grigat 2003-2019

0.11.8.5 Löschoperation in Heap h mit Sift

deletemin(h)

```
1  Ausgabe der Wurzel von h, d.h. Knoten mit minimalem Schlüssel von h
2  Kopieren des Eintrags p der letzten besetzten Position in die Wurzel von h
3  Löschen des alten Knotens p an der letzten besetzten Position
4
5  Absinken der Kopie von p beginnend bei Wurzel von h
6  Seien jeweils p Wurzel und p_l, p_r die Söhne links und rechts,
7  min(p_l,p_r) liefere Knoten mit dem kleineren Schlüssel
8
9  while (p_l oder p_r existieren)
10                    and ( (key(p) > key(p_l)) or (key(p) > key(p_r)) )
11 {
12         vertausche p mit min(p_l, p_r) (=: Sift)
13 }
```

0.11.8.6 Korrektheit des Algorithmus zu Entnahme des Minimums

Beweis der Korrektheit von *deletemin*

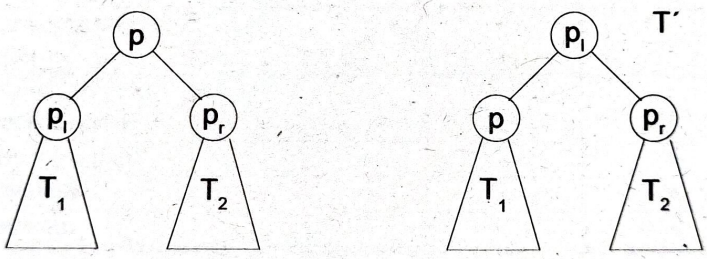


Abbildung 98: Vertauschung bei Entnahme des Minimums

- p ist das an die Wurzel getauschte Element aus der letzten Position nach Entnahme des Minimums
- p_l ist minimal bzgl. T_1, p_r ist minimal bzgl. T_2
- Annahme: Vertauschung von p und p_l notwendig, also:
  - Es gilt $\text{key}(p_l) < \text{key}(p)$ und $\text{key}(p_l) \leq \text{key}(p_r)$
  - $\text{key}(p_l)$ ist minimal bzgl. $\text{key}(p)$, $\text{key}(p_r)$, T_1 und T_2
  - Falls $\text{key}(p)$ nicht minimal bzgl. T_1 ist, wird *Sift(p)* in T_1 durchgeführt
  - *Sift(p)* wird wiederholt, bis gilt: $\text{key}(p) \leq \text{key}(p_l)$ und $\text{key}(p) \leq \text{key}(p_r)$

© R.-R. Grigat 2003-2019

0.11.8.7 Beispiel: Löschoperation

Beispiel: deletemin(h)

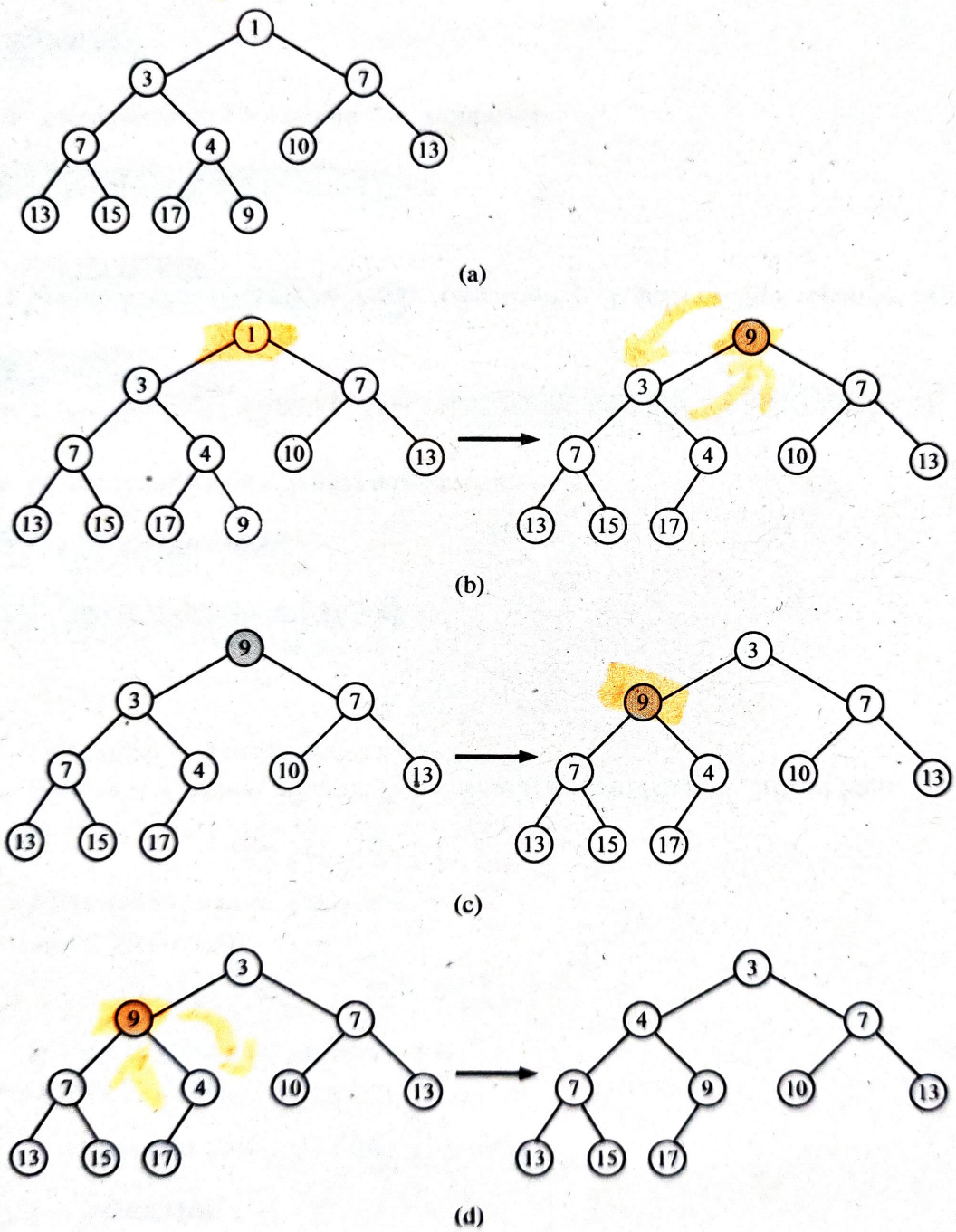


Abbildung 99: Beispiel: Entnahme des minimalen Elements

© R.-R. Grigat 2003-2019

0.11.8.8 Komplexität

- Der Baum ist balanciert
- Einfüge- und Löschoperation werden in $O(\log n)$ Zeit realisiert

Beispiel: KP36_12_heap

0.11.9 Übersicht Baumstrukturen

	Binärer Suchbaum	AVL Baum	Heap
Binärbaum	✓	✓	✓
Vollständig	-	-	✓
Links-vollständig	-	-	✓
balanciert (keine Entartung zu Liste)	-	✓	✓
Sortiert in Inorder-Reihenfolge	✓	✓	-
Partielle Ordnung	-	-	✓
Implementiert z.B. den Datentyp	Dictionary	Dictionary	Priority Queue

Tabelle 24: Vergleich der Baumstrukturen

© R.-R. Grigat 2003-2019